



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ
РАБОТЕ НА ТЕМУ:**

***«Автоматическая оптимизация планов выполнения
SQL-запросов»***

Студент ИУ9-81Б

(Группа)

(Подпись, дата)

Старовойтов А. И.

(Фамилия И.О.)

Научный руководитель

(Подпись, дата)

Непейвода А. Н.

(Фамилия И.О.)

Нормоконтролёр

(Подпись, дата)

Алексеев С. М.

(Фамилия И.О.)

АННОТАЦИЯ

Работа посвящена разработке оптимизатора SQL-запросов для модельной реляционной СУБД. Цель работы состоит в реализации оптимизатора на основе архитектуры Cascades и создании основы для его дальнейшего улучшения за счет сопоставления порождаемых физических планов с планами промышленных систем.

В ходе исследования рассмотрены архитектуры оптимизаторов System R, Volcano и Cascades, формализовано поддерживаемое подмножество SQL и соответствующее ему подмножество реляционной алгебры. При реализации использованы структура Мемо, правила логических преобразований и физических реализаций, стоимостная модель, учет требуемых физических свойств, а также метод ветвей и границ для отсеечения неперспективных вариантов планов. Для оценки работы системы применены модульное тестирование, генерация SQL-запросов, бенчмарки физических операторов и дифференциальное сравнение планов с Microsoft SQL Server.

В результате разработана модельная СУБД, работающая с CSV-файлами и поддерживающая основные операторы реляционной алгебры, а также расширяемый оптимизатор, способный строить физические планы с учетом стоимости и физических свойств результата. Получены стоимостные формулы операторов, подтвержден прирост производительности относительно наивного плана и подготовлены инструменты для поиска случаев, в которых пространство поиска оптимизатора требует расширения. Практическая значимость работы заключается в возможности использовать реализованную систему как основу для экспериментального исследования методов оптимизации запросов и итеративного развития набора правил оптимизатора.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Исследовательская часть	6
1.1 Процесс исполнения SQL-запроса	6
1.2 Реляционная алгебра	8
1.3 Преобразование синтаксического дерева в реляционную алгебру	10
1.4 Поиск оптимального плана	12
1.5 Обзор архитектур оптимизаторов	16
1.6 Архитектура Cascades	17
1.6.1 Группы эквивалентности и выражения	17
1.6.2 Алгоритм поиска	18
1.7 Правила трансформации и реализации	19
1.7.1 Метод ветвей и границ	20
1.7.2 Оценка кардинальности	22
1.7.3 Оценка стоимости	22
1.8 Дифференциальный анализ физических планов	23
2 Конструкторская часть	27
2.1 Модули синтаксического анализа и построения логиче- ского представления	27
2.2 Модули физического хранения и исполнения	28
2.3 Модуль стоимостной оптимизации планов	29
3 Технологическая часть	33
3.1 Модуль синтаксического анализа	33
3.2 Модуль оптимизации запросов	34
3.2.1 Правила трансформации и реализации	36
3.2.2 Алгоритм поиска	36
3.2.3 Метод ветвей и границ	37
3.2.4 Физические свойства	38
3.2.5 Формирование физического плана	39

3.3	Модуль физического хранения данных	40
3.4	Модуль исполнения физических планов	40
3.5	Руководство пользователя	42
4	Аналитическая часть	45
4.1	Дифференциальный анализ и тестирование производи- тельности	46
4.2	Калибровка стоимостной модели	48
	ЗАКЛЮЧЕНИЕ	50
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	51
	ПРИЛОЖЕНИЕ А	53

ВВЕДЕНИЕ

В эпоху стремительного роста объемов данных системы управления базами данных (СУБД) являются важной частью информационных систем. По данным аналитических исследований[1], объем мирового рынка СУБД ежегодно увеличивается, что обусловлено как расширением круга решаемых задач, так и усложнением структуры обрабатываемых данных и увеличением их объема. Реляционные базы данных, работающие с различными диалектами языка SQL, продолжают занимать доминирующее положение в сегменте обработки транзакций и аналитических запросов.

Одним из определяющих факторов конкурентоспособности между различными реализациями СУБД является качество оптимизатора запросов. Оптимизатор представляет собой компонент, ответственный за преобразование декларативного SQL-запроса в эффективный физический план исполнения. Именно от качества работы оптимизатора в значительной степени зависит производительность всей системы: выбор неоптимального плана исполнения может привести к увеличению времени выполнения запроса в десятки раз относительно оптимального случая. Также оптимизатор запросов считается наиболее сложной составной частью любой СУБД. Задача нахождения оптимального плана является NP-полной в общем случае, что обуславливает необходимость применения эвристических методов и алгоритмов ограниченного перебора.

Архитектура оптимизаторов Cascades используется в таких промышленных системах, как Microsoft SQL Server, Apache Orca, а также в ряде исследовательских проектов, включая Columbia и CockroachDB. Преимущества этой архитектуры заключаются в расширяемости, модульности, а также понятном и эффективном алгоритме поиска, допускающим параллельность исполнения.

Целью данной работы является разработка и реализация оптимизатора подмножества SQL-запросов на основе архитектуры Cascades, а также создание каркаса для его итеративного улучшения путем сравнения с промышленными реализациями.

Для достижения поставленной цели прежде всего требуется изучить архитектуру оптимизаторов Cascades и на ее основе проработать собственную.

После выбора архитектурной основы необходимо формализовать поддерживаемое подмножество SQL и соответствующее ему подмножество реляционной алгебры. Эта задача определяет границы рассматриваемых запросов, их семантику и внутреннее логическое представление, используемое на этапе оптимизации.

Следующим шагом является реализация структуры данных Метод и алгоритма поиска оптимального физического плана на основе архитектуры Cascades. В рамках этой задачи требуется обеспечить хранение эквивалентных логических выражений, повторное использование найденных вариантов планов и выбор плана с минимальной оценочной стоимостью.

Чтобы сформировать пространство поиска оптимизатора, требуется разработать набор правил трансформации и реализации для основных операторов реляционной алгебры. Правила трансформации должны порождать логически эквивалентные планы, а правила реализации сопоставлять логическим операторам допустимые физические операторы.

Для оптимизации алгоритма поиска нужно реализовать метод ветвей и границ. Этот метод должен использовать оценки стоимости для отсека заведомо неоптимальных планов и уменьшения объема пространства поиска, исследуемого оптимизатором.

Для последующего развития системы также требуется реализовать метод дифференциального анализа физических планов. Он должен обеспечивать автоматизированный поиск примеров неоптимальных планов путем сравнения планов разрабатываемого оптимизатора с планами промышленных СУБД, а найденные различия должны использоваться для уточнения правил оптимизации.

1 Исследовательская часть

Система управления базами данных представляет собой программный комплекс, обеспечивающий хранение, извлечение и модификацию структурированных данных. Несмотря на разнообразие существующих реализаций, архитектура большинства реляционных СУБД включает ряд типовых компонентов, взаимодействие которых обеспечивает выполнение пользовательских запросов[2].

К основным компонентам СУБД относятся (рисунок 1):

- транспортный уровень, принимающий клиентские подключения и запросы;
- синтаксический анализатор (парсер), выполняющий разбор текста SQL-запроса и построение синтаксического дерева;
- оптимизатор запросов, преобразующий логическое представление запроса в эффективный физический план исполнения;
- исполнитель, непосредственно реализующий физический план;
- менеджер хранения данных, управляющий размещением данных на физических носителях;
- менеджер транзакций и менеджер блокировок, обеспечивающие гарантии транзакций;
- менеджер страниц памяти, отвечающий за кэширование страниц данных в оперативной памяти.

1.1 Процесс исполнения SQL-запроса

Процесс исполнения SQL-запроса в реляционной СУБД проходит несколько последовательных этапов.

1. Синтаксический анализ — запроса проходит лексический и синтаксический разбор. Результатом данного этапа является синтаксическое дерево, узлы которого соответствуют конструкциям языка SQL: операторам SELECT, FROM, WHERE, JOIN, GROUP BY, ORDER BY и другим. На этом же этапе выполняется семантический анализ: проверяется суще-



Рисунок 1 – Архитектура реляционной системы управления базами данных

ствование указанных таблиц и столбцов, разрешаются имена объектов и определяются типы выражений.

2. Оптимизация — синтаксическое дерево преобразуется в логический план запроса, выраженный в терминах реляционной алгебры. Оптимизатор исследует пространство эквивалентных логических планов и для каждого из них рассматривает возможные физические реализации операторов. С помощью модели стоимости оптимизатор оценивает затраты на выполнение каждого плана и выбирает план с наименьшей оценочной стоимостью. Результатом этого этапа является физический план исполнения. Под этим понимается дерево физических операторов.

ров с указанием конкретных алгоритмов соединения, методов доступа к данным и порядка операций.

3. Исполнение физического плана — движок выполнения реализует физический план, порождая потоки кортежей. Данные извлекаются из хранилища, обрабатываются операторами плана и формируют результирующий набор, возвращаемый пользователю.

Сформулируем задачу, которую решает оптимизатор: по запросу, представленному в форме дерева операторов реляционной алгебры, построить эквивалентный и оптимальный, с точки зрения необходимых для выполнения ресурсов, план исполнения. Итоговый план является деревом, вершины которого помечены конкретными указаниями на алгоритмы, реализующими один или несколько операторов реляционной алгебры. Эквивалентность означает, что итоговый физический план и исходный запрос при исполнении на любом наборе данных порождают одинаковые отношения с точностью до требуемых свойств, таких как сортировка.

Запрос в форме реляционной алгебры также называют логическим планом, а операторы реляционной алгебры — логическими операторами.

Для каждого логического оператора существует несколько возможных физических реализаций. Например, оператор соединения $\bowtie$ может быть реализован как соединение вложенными циклами, хеш-соединение или соединение слиянием. Аналогично, оператор доступа к данным может быть реализован как последовательное сканирование таблицы или сканирование индекса. Еще одним важным примером является эквивалентность сканирования индекса с предикатом и двух логических операторов, примененных последовательно: доступ к данным и фильтрация по тому же условию.

1.2 Реляционная алгебра

Реляционная алгебра представляет собой формальный язык для описания операций над отношениями реляционной базы данных. В отличие от декларативного SQL, реляционная алгебра является процедурной, так как она определяет конкретную последовательность операций, необходимых для получения результата.

Отношение определяется как конечное множество кортежей, каждый из которых представляет собой упорядоченный набор значений атрибутов:

$$R \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n).$$

Среди основных операторов реляционной алгебры выделяют следующие[3].

1. Фильтрация $\sigma_p(R)$ — возвращает подмножество кортежей отношения R , удовлетворяющих предикату p . Например, $\sigma_{\text{age} > 30}(\text{Employee})$ вернет все кортежи из отношения Employee, для которых значение атрибута age больше 30.
2. Проекция $\pi_{A_1, \dots, A_k}(R)$ — формирует новое отношение, содержащее только перечисленные атрибуты.
3. Декартово произведение $R \times S$ — формирует отношение, каждый кортеж которого является конкатенацией кортежа из R и кортежа из S , при этом результат содержит $|R \times S| = |R| \cdot |S|$ элементов.
4. Соединение $R \bowtie_p S$ — комбинирует кортежи двух отношений на основании предиката p . Внутреннее соединение определяется как $R \bowtie_{R.a=S.b} S = \sigma_{R.a=S.b}(R \times S)$.
5. Объединение $R \cup S$ — возвращает все кортежи, принадлежащие хотя бы одному из двух совместимых по схеме отношений.
6. Разность $R - S$ — выбирает кортежи, принадлежащие R , но отсутствующие в S .
7. Агрегация $\gamma_{G; f}(R)$ — разбивает отношение R на непересекающиеся множества по набору атрибутов G и вычисляет агрегированные значения функции f по каждому такому множеству. Примеры функций: COUNT, SUM, AVG, MIN, MAX.
8. Apply $R \mathcal{A} S(t)$ — для каждого кортежа t отношения R вычисляет параметризованное им отношение $S(t)$ и конкатенирует t с каждым кортежем результата:

$$R \mathcal{A} S(t) = \bigcup_{t \in R} \{t\} \times S(t).$$

В отличие от соединения, правая сторона зависит от текущего кортежа левой и используется для представления коррелированных подзапросов.

1.3 Преобразование синтаксического дерева в реляционную алгебру

Полученное после лексического и синтаксического разбора абстрактное синтаксическое дерево преобразуется в выражение в форме реляционной алгебры. Этот процесс определяется структурной индукцией по дереву запроса. Введём оператор конверсии $\llbracket \cdot \rrbracket$, сопоставляющий узлу синтаксического дерева его алгебраическое представление. Тогда правила трансляции записываются в виде правил вывода. Если для всех поддеревьев известны алгебраические представления, то алгебраическое представление составной конструкции выражается через них.

1. Базовый случай. Для ссылки на таблицу T выполняется $\llbracket T \rrbracket = T$.
2. Блок SELECT-FROM-WHERE. Пусть $\bar{T} = T_1, \dots, T_n$ — ссылки на таблицы или вложенные подзапросы, p — предикат фильтрации, $\bar{a} = a_1, \dots, a_k$ — список выражений проекции. Тогда

$$\frac{\llbracket T_1 \rrbracket = E_1 \quad \dots \quad \llbracket T_n \rrbracket = E_n}{\llbracket \text{SELECT } \bar{a} \text{ FROM } \bar{T} \text{ WHERE } p \rrbracket = \pi_{\bar{a}}(\sigma_p(E_1 \times \dots \times E_n))}.$$

3. Группировка. Пусть $\bar{g} = g_1, \dots, g_m$ — список группирующих атрибутов, $\bar{f} = f_1, \dots, f_l$ — список агрегирующих функций. Тогда

$$\frac{\llbracket T_1 \rrbracket = E_1 \quad \dots \quad \llbracket T_n \rrbracket = E_n}{\llbracket \text{SELECT } \bar{g}, \bar{f} \text{ FROM } \bar{T} \text{ WHERE } p \text{ GROUP BY } \bar{g} \rrbracket = \gamma_{\bar{g}, \bar{f}}(\sigma_p(E_1 \times \dots \times E_n))}.$$

4. Вложенные подзапросы. Пусть Q — внешний блок, а $Q'(t)$ — коррелированный подзапрос, зависящий от кортежа t из $\llbracket Q \rrbracket$. Тогда

$$\frac{\llbracket Q \rrbracket = E \quad \llbracket Q'(t) \rrbracket = E'(t)}{\llbracket Q \text{ с подзапросом } Q' \rrbracket = E \mathcal{A} E'(t)},$$

где $\mathcal{A}$ — оператор Apply, который впоследствии может быть удалён процедурой декорреляции.

5. Сортировка. Конструкция ORDER BY не выражается в виде операторов реляционной алгебры. Вместо этого она задает требуемое физическое свойство упорядоченности результата, учитываемое на этапе выбора физического плана.

Приведём пример применения этих правил к SQL-запросу из листинга 1. Трансляция этого запроса в реляционную алгебру выполняется по следующим шагам:

Листинг 1: Запрос с коррелированным подзапросом.

```
SELECT e.name
FROM Employee e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM Employee
    WHERE dept = e.dept
)
```

1. По базовому правилу оба вхождения таблицы остаются как есть $\llbracket \text{Employee} \rrbracket = \text{Employee}$.
2. Внутренний подзапрос $Q'(e) = (\text{SELECT AVG(salary) FROM ... WHERE ...})$ транслируется по правилу группировки, где $\bar{g} = \emptyset$, $\bar{f} = \text{AVG(salary)}$:

$$\llbracket Q'(e) \rrbracket = \gamma_{\emptyset; \text{AVG(salary)}}(\sigma_{\text{dept}=e.\text{dept}}(\text{Employee})).$$

Подзапрос коррелирован, так как зависит от атрибута $e.\text{dept}$ внешнего кортежа.

3. По правилу вложенных подзапросов результат $\llbracket Q'(e) \rrbracket$ присоединяется к внешнему блоку оператором $\mathcal{A}$, добавляя к каждому кортежу столбец c с вычисленным значением средней зарплаты.
4. Оставшаяся часть внешнего запроса переводится по правилу SELECT-FROM-WHERE с $\bar{a} = (e.\text{name})$ и $p = (e.\text{salary} > c)$.

Итоговое алгебраическое выражение имеет вид

$$\pi_{e.\text{name}}(\sigma_{e.\text{salary} > c}(\text{Employee } e \mathcal{A} \gamma_{\emptyset; \text{AVG(salary)} \rightarrow c}(\sigma_{\text{dept}=e.\text{dept}}(\text{Employee}))))).$$

Соответствующее дерево операторов изображено на рисунке 2.

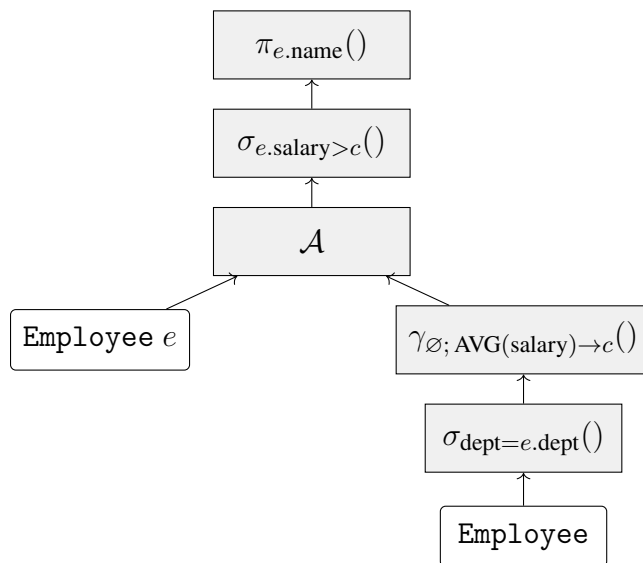


Рисунок 2 – Дерево реляционной алгебры для запроса из листинга 1.

Преобразование в форму реляционной алгебры необходимо по нескольким причинам. Во-первых, это позволяет использовать формализованный набор правил эквивалентности, который позволяют оптимизатору корректно преобразовывать план запроса без изменения семантики. Во-вторых, алгебраическое представление позволяет единообразно обрабатывать различные синтаксические формы SQL-запросов, порождающие одинаковые логические планы. Наконец, разделение на логические и физические операторы позволяет оптимизатору независимо исследовать порядок операций и алгоритмы их реализации.

1.4 Поиск оптимального плана

Эквивалентность планов, исследуемых в процессе поиска оптимального, обеспечивается применением алгебраических тождеств, например ассоциативности и коммутативности соединений. Правила эквивалентности реляционной алгебры подробно рассматриваются в разделе 1.7.

Оптимальность запроса принято оценивать с помощью функции стоимости $\text{cost}: \mathcal{P}_L \rightarrow \mathbb{R}_{\geq 0}$ [4], где $\mathcal{P}_L$ — множество эквивалентных физических планов. Это отображение оценивает затраты на исполнение плана исходя из конкретных реализаций алгоритмов, а также основываясь на эвристиках и статистике о данных в отношениях. Другими словами, цель оптимизатора состоит

в том, чтобы для входного запроса найти план $P^* \in \mathcal{P}_L$, минимизирующий cost:

$$P^* = \arg \min_{P \in \mathcal{P}_L} \text{cost}(P).$$

Одной из подзадач в рамках построения оптимального плана является определение порядка выполнения соединений. Благодаря правилам эквивалентности, таким как ассоциативность и коммутативность соединения (раздел 1.7), одно и то же множество соединяемых отношений допускает большое количество эквивалентных порядков вычисления. Различные порядки имеют кардинально разные стоимости, ввиду особенностей алгоритмов, реализующих соединение. Для n отношений количество таких порядков выражается как $O(n! \cdot C_{n-1}) = O(\frac{(2n-2)!}{(n-1)!})$ [5], где C_k — k -е число Каталана. Таким образом, пространство поиска растет экспоненциально с увеличением числа соединяемых таблиц.

Поскольку полный перебор $\mathcal{P}_L$ для достаточно больших n невозможен, на практике речь идёт о поиске плана с приемлемо малой стоимостью в определенном подмножестве пространства поиска. Именно эта задача и определяет архитектуру современных оптимизаторов: расширяемое описание пространства поиска через правила преобразования, эффективный алгоритм его исследования и стоимостная модель, позволяющая отсекаать заведомо неоптимальные ветви.

Выбор плана исполнения запроса может оказывать огромное влияние на объем требуемых вычислительных ресурсов. В качестве примера оценим количество чтений страниц с диска для различных физических операторов. Пусть имеется три отношения: таблица `customer` с $|C| = 10^5$ строк, таблица `orders` с $|O| = 10^7$ строк и таблица `lineitem` с $|L| = 5 \cdot 10^7$ строк. Размер страницы — 8 КБ. В одну страницу помещается 100 строк `customer`, 80 строк `orders` и 50 строк `lineitem`. Соответственно, общее число страниц составляет $P_C = 10^3$, $P_O = 1,25 \cdot 10^5$, $P_L = 10^6$.

Для запроса из листинга 2 рассмотрим два альтернативных плана (рисунок 3):

Листинг 2: Соединение customer, orders и lineitem.

```
SELECT *
FROM customer c
JOIN orders o ON c.c_custkey = o.o_custkey
JOIN lineitem l ON o.o_orderkey = l.l_orderkey
```

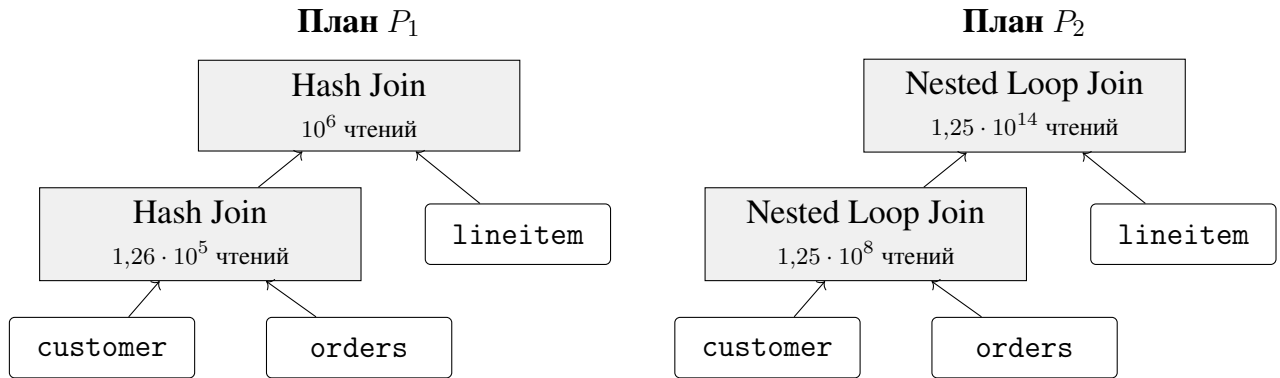


Рисунок 3 – Сравнение планов для запроса из листинга 2.

- План P_1 : $(\text{customer} \bowtie \text{orders}) \bowtie \text{lineitem}$ с реализацией обоих соединений через Hash Join, для чего сначала полностью считываются $P_C + P_O = 1,26 \cdot 10^5$ страниц для построения хеш-таблицы, после чего промежуточный результат размером 10^7 строк соединяется с `lineitem`, требуя считывания ещё $P_L = 10^6$ страниц. В сумме — порядка $1,126 \cdot 10^6$ операций чтения с диска.
- План P_2 : тот же запрос, но реализованный как соединение с помощью вложенных циклов без использования индексов: для каждой строки `customer` полностью читается таблица `orders`, для каждой результирующей пары — `lineitem`. Совокупный объём ввода-вывода оценивается величиной $P_C \cdot P_O \cdot P_L \approx 1,25 \cdot 10^{14}$ операций чтения, что на восемь порядков превышает P_1 .

На еще одном примере запроса из листинга 3 рассмотрим влияние логических преобразований. Отношение `Emp` содержит 10,000 кортежей, что эквивалентно 1,000 страницам. `Dept` включает 500 записей и 50 страниц соответственно. Атрибут `Emp.did` является внешним ключом к `Dept.did`, поэтому каждому кортежу из `Dept` соответствует $10,000/500 = 20$ из `Emp`. Пусть на `Emp.did` и

Листинг 3: Пример запроса с проекцией, фильтрацией и соединением.

```
SELECT DISTINCT ename
FROM Emp E JOIN Dept D ON E.did = D.did
WHERE D.dname = 'Toy'
```

Dept.dname построены индексы, а в промежуточную страницу помещается 5 кортежей.

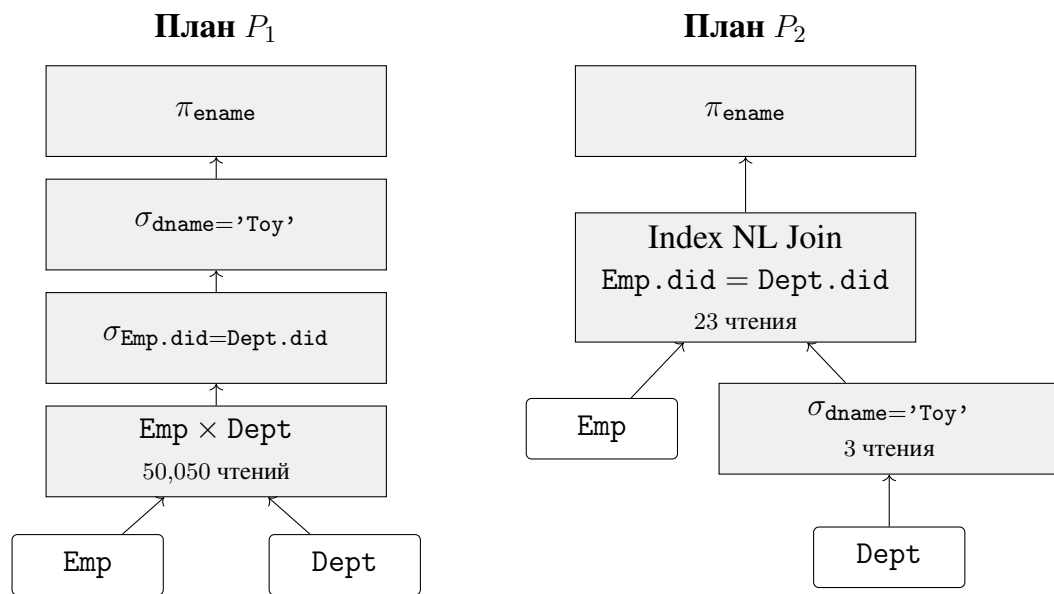


Рисунок 4 – Сравнение планов для запроса из листинга 3.

Рассмотрим два плана (рисунок 4):

- План P_1 соответствует прямому преобразованию запроса в реляционную алгебру, выполненному по алгоритму из раздела 1.3. Если рассматривать стоимость исполнения такого плана в модели итераторов, то каждый оператор запрашивает у дочернего следующий кортеж через вызов `next()`, поэтому σ и π работают в режиме конвейера и не инициируют обращений к диску. Вся стоимость ввода-вывода сосредоточена в операторе $\times$, реализованном через вложенные циклы с Dept в роли внешней таблицы: Dept читается один раз, а Emp перечитывается полностью для каждой страницы Dept:

$$50 + 50 \times 1,000 = 50,050 \text{ чтений.}$$

- План P_2 является оптимизированным вариантом, где применено правило проталкивания предиката и декартово произведение заменено на соединение по предикату. Причем для доступа к таблице Dept применяется сканирование по индексу, что дает всего 3 чтения с диска. Соединение реализуется через вложенные циклы с использованием индекса, что в итоге дает:

$$3 + (3 + 20) = 26 \text{ чтений.}$$

Оба приведенных примера наглядно демонстрируют, что оптимизатор отвечает за принципиальную возможность исполнения сложных аналитических запросов за приемлемое время. В первом случае разница между физическими планами составляет восемь порядков: если P_1 завершается за секунды, то наивный P_2 потребует десятков лет. Во втором случае даже единственное логическое преобразование, в виде проталкивания предиката, снижает число чтений с диска в 1,900 раз. Именно поэтому качественный оптимизатор является конкурентным преимуществом и одной из самых важных частей СУБД.

1.5 Обзор архитектур оптимизаторов

System R, разработанный в IBM Research в конце 1970-х годов [4], стал первым стоимостным оптимизатором. Его основная идея состоит в восходящем динамическом программировании. Оптимизатор сначала выбирает лучшие методы доступа к отдельным отношениям, затем строит лучшие планы для соединений из двух, трех и большего числа отношений. Преимущество System R заключается в простоте и предсказуемости поиска, а также в учете стоимости ввода-вывода, процессорных затрат и физических свойств операторов. Недостатки связаны с ограниченной расширяемостью, ориентацией на левосторонние деревья соединений и независимой оптимизацией вложенных подзапросов.

Volcano [6—8] развивает стоимостный подход System R, но делает оптимизатор расширяемым за счет правил и структуры Memo. В отличие от System R, пространство эквивалентных выражений задается не жестко встроенным алгоритмом, а набором правил трансформации и реализации. Поиск выполняется

нисходящим динамическим программированием и может учитывать требуемые физические свойства. Преимущество Volcano состоит в модульности и возможности добавлять новые преобразования и операторы без переписывания всего оптимизатора. Основным недостатком заключается в разделении поиска на фазу генерации и фазу стоимостного анализа: предварительное раскрытие всех классов эквивалентности может приводить к избыточной работе.

1.6 Архитектура Cascades

Cascades [9; 10] является развитием Volcano и сохраняет его ключевые идеи: правила, Мемо, физические свойства и стоимостный поиск. Главное отличие Cascades состоит в том, что генерация альтернатив и стоимостная оптимизация не разделяются на две независимые фазы, а чередуются через систему задач. Благодаря этому оптимизатор может порождать только те альтернативы, которые нужны для текущего поиска, раньше применять отсечения по стоимости и удобнее распараллеливать независимые ветви. Недостатком Cascades является более сложная реализация, так как требуется управлять состоянием групп, задачами поиска, правилами и физическими свойствами.

В следующих разделах подробно рассматриваются основные элементы архитектуры оптимизаторов Cascades.

1.6.1 Группы эквивалентности и выражения

Группа представляет собой множество выражений, которые эквивалентны с точки зрения логического результата. Например, для отношений A , B и C выражения $A \bowtie (B \bowtie C)$; $(A \bowtie B) \bowtie C$ при выполнении условий ассоциативности соединения принадлежат одной группе, поскольку возвращают одинаковый результат. Причем физические реализации этих выражений могут отличаться, а стоимость может зависеть от кардинальности промежуточных результатов.

Групповое выражение хранит оператор и ссылки на дочерние группы. Например, выражение $Join(G_1, G_2)$ не указывает конкретное дерево для левого и правого входов, а ссылается на группы G_1 и G_2 , каждая из которых может со-

Листинг 4: Пример запроса.

```
SELECT * FROM A JOIN B ON p_1 JOIN C ON p_2;
```

держат множество альтернатив. Поэтому одно групповое выражение кодирует комбинацию многих конкретных деревьев, храня эту информацию компактно.

Мето является основной структурой данных для хранения групп эквивалентных выражений в Cascades. Она выполняет функции устранения дублирующихся выражений, хранит альтернативы, фиксирует результаты оптимизации под разными физическими свойствами и является точкой синхронизации для задач поиска.

На рисунке 9 показан пример Мето для запроса из листинга 4. Например, группа G_5 представляет результат соединения трех отношений, в ней находятся два логически эквивалентных выражения с разной ассоциацией соединений и четыре физические реализации для них. Входными данными для операторов являются группы, поэтому общие подвыражения не дублируются.

При добавлении нового выражения Мето выполняет проверку на дубликаты. Если выражение уже существует в некоторой группе, оно не добавляется повторно. В случае, когда алгоритм в процессе работы обнаруживает эквивалентность двух ранее различных групп, они эти группы объединяются в одну. При этом требуется обновление ссылок и состояния правил.

1.6.2 Алгоритм поиска

Cascades организует оптимизацию как выполнение набора задач. Задача может исследовать группу, исследовать выражение, применить правило, реализовать логический оператор, оптимизировать дочернюю группу под заданное физическое свойство или построить вспомогательный оператор для обеспечения свойства. Такой подход позволяет гибко управлять порядком работы и откладывать дорогостоящие действия до тех пор, пока они не станут необходимыми, а также разбивать поиск на несколько этапов и распараллеливать

исполнение независимых веток. При этом однопоточные реализации хранят задачи в обычной очереди.

На рисунке 8 показана схема взаимодействия задач в оптимизаторе, а на листингах 1-2 приведен псевдокод алгоритма поиска в архитектуре Cascades [11].

Алгоритм реализуется в виде управляемого стеком поиска в пространстве эквивалентных планов: сначала запускается оптимизация корневой группы, `OptimizeGroup` в свою очередь исследует группу, а затем оптимизирует каждое ее логическое выражение. Исследование выполняется через `ExploreGroup` и `ExploreExpression`, которые сохраняют в Мемо новые логические выражения и группы с помощью правил преобразования. После этого `OptimizeExpression` применяет правила реализации, превращая логические выражения в физические. Для каждого физического варианта `ApplyImplementation` проверяет локальную стоимость и передает план в `OptimizeInputs`, где последовательно оптимизируются дочерние группы, накапливается полная стоимость и обновляется информация о лучшем плане. Параметр `limit` используется для отсеечения заведомо дорогих вариантов.

1.7 Правила трансформации и реализации

Правило в оптимизаторе Cascades состоит из образца, условия применимости и алгоритма применения. Образец описывает форму логического выражения, к которому может быть применено правило. Условие применимости проверяет дополнительные требования: отсутствие внешних ссылок, тип соединения, сохранение семантики NULL, совместимость свойств и другие ограничения.

Формально правило записывается в виде

$$r : \frac{\text{pattern}(e)}{\text{condition}(e)} \Rightarrow \text{substitute}(e),$$

где e — исходное выражение, pattern — структурный образец, condition — предикат применимости, substitute — выражение-замена. Для правил транс-

формации результатом применения является новое логическое выражение в группе, а для правил реализации физическое выражение в той же группе.

Правила трансформации задают эквивалентные преобразования логических выражений и тем самым определяют пространство планов, исследуемое оптимизатором. Наиболее важные из них представлены в таблице 4.

Определение 1. *Null-отбрасывающим называется предикат, который при замене всех атрибутов соответствующей стороны на NULL принимает значение, отличное от TRUE.*

Определение 2. *Конъюнкты $\theta_1 \wedge \theta_2$ называются перераспределяемыми, если их можно разбить на две группы θ'_1 и θ'_2 такие, что атрибуты θ'_2 принадлежат $attrs(S) \cup attrs(T)$, а конъюнкты с атрибутами из $attrs(R)$ остаются в θ'_1 .*

Определение 3. *Функция агрегации называется разделяемой, если она представима композицией частичного агрегата и финального комбинатора. Например, SUM раскладывается в сумму частичных сумм, AVG — в пару SUM и COUNT.*

Правила реализации переводят логические операторы в физические. Используемые в настоящей работе правила собраны в таблице 6.

Физические свойства в Cascades обрабатываются через требуемые и предоставляемые свойства. Если родительский оператор требует вход, отсортированный по атрибуту a , а выбранный дочерний план такого порядка не предоставляет, оптимизатор может добавить обеспечивающий оператор Sort.

Обеспечивающие операторы не изменяют логический результат, но меняют физические свойства и увеличивают стоимость. Поэтому они рассматриваются наравне с другими физическими альтернативами. Например, план с хеш-соединением и последующей сортировкой может конкурировать с планом на основе соединения слиянием, которое сохраняет порядок результата.

1.7.1 Метод ветвей и границ

Метод ветвей и границ используется для сокращения пространства поиска и, как следствие, неасимптотического улучшения скорости работы алгоритма.

В контексте Cascades ветвями являются альтернативные выражения и комбинации физических реализаций дочерних групп, а границей является текущая лучшая стоимость плана для заданной группы и требуемых физических свойств. Если нижняя оценка стоимости частично построенного плана уже превышает текущую лучшую, дальнейшее исследование по этой ветке не имеет смысла.

Пусть для группы G и требуемого свойства P уже найден план стоимости C^* . Рассматривается физическое выражение e , имеющее локальную стоимость $C_{local}(e)$ и дочерние группы $G_1, \dots, G_k$. Если даже минимально возможная оценка дочерних групп приводит к неравенству

$$C_{local}(e) + \sum_{i=1}^k LB(G_i, P_i) \geq C^*,$$

то выражение e может быть отсечено. Здесь LB обозначает нижнюю границу стоимости оптимизации дочерней группы под требуемым свойством P_i . Чем точнее нижняя граница, тем эффективнее отсечение.

На практике в качестве $LB(G_i, P_i)$ принимается лучшая стоимость, найденная для пары (G_i, P_i) на текущий момент работы алгоритма. Поскольку оптимизация группы является рекурсивным процессом, к моменту отсечения для (G_i, P_i) уже могут быть найдены некоторые планы, и их минимальная стоимость служит нижней границей. Если же группа G_i под свойством P_i ещё не рассматривалась, используется тривиальная граница $LB = 0$, или, при условии наличия минимальной стоимости на кортеж, можно принимать $LB(G_i, P_i) = |G_i| \cdot minCostPerRow$.

Требуемые свойства дочерних групп P_i определяются оператором выражения e совместно с требованием родителя P . Каждый физический оператор описывает, какие свойства он ожидает от своих входов: например, оператор сортирующего слияния требует упорядоченности обоих входов по соответствующим ключам, тогда как хеш-соединение не предъявляет требований к порядку. Таким образом, при спуске по дереву поиска каждый оператор транслирует глобальное требование P в конкретные требования $P_1, \dots, P_k$ к своим дочерним группам.

1.7.2 Оценка кардинальности

Оценка кардинальности является процессом предсказания числа кортежей, возвращаемых оператором.

Для выборки используется селективность предиката $sel(p)$, после чего кардинальность результата оценивается как

$$|\sigma_p(R)| = |R| \cdot sel(p).$$

Для соединения на основе предиката равенства независимых атрибутов часто применяется эвристика:

$$|R \bowtie_{R.a=S.b} S| \approx \frac{|R| \cdot |S|}{\max(NDV(R.a), NDV(S.b))},$$

где NDV — число различных значений атрибута. Такая оценка удобна, но может быть грубой, если данные имеют корреляции, сильный перекос или сложные зависимости между атрибутами. Экспериментальные исследования показывают, что ошибки кардинальности часто являются одной из основных причин выбора неудачных планов [12].

Для повышения качества оценок используются гистограммы, статистики по многоколоночным ключам, сведения об уникальности, функциональные зависимости, ограничения на схему, а в современных исследованиях также модели машинного обучения.

1.7.3 Оценка стоимости

Функция стоимости сопоставляет физическому плану численную оценку затрат на его исполнение. Классическим считается представление стоимости в виде взвешенной суммы операций чтения и записи страниц на диск, а также числа обработанных кортежей и операций над ними [6; 9]:

$$Cost(p) = w_{io} \cdot IO(p) + w_{cpu} \cdot CPU(p).$$

Коэффициенты w_{io} , w_{cpu} зависят от конкретной аппаратной конфигурации.

Стоимость является аддитивной и вычисляется рекурсивно по дереву физического плана:

$$Cost(node) = LocalCost(node) + \sum_{child \in children(node)} Cost(child).$$

Локальная стоимость оператора определяется его типом и кардинальностью его входов. Пусть $|R|$ — кардинальность отношения R , b — блочный фактор (число кортежей на странице), тогда $P(R) = \lceil |R|/b \rceil$ — число страниц, занимаемых R ; c_θ — стоимость вычисления предиката θ на одной паре кортежей; σ_θ — селективность предиката θ (доля кортежей входа, удовлетворяющих θ); M — число страниц буфера, доступных оператору. Формулы локальной стоимости физических операторов приведены в таблице 1.

Таблица 1 – Локальная стоимость физических операторов.

Оператор	$IO(op)$	$CPU(op)$
SeqScan(T)	$P(T)$	$ T $
IndexScan(T, θ)	$\lceil \sigma_\theta \cdot P(T) \rceil$	$\sigma_\theta \cdot T $
Filter(R, θ)	0	$ R \cdot c_\theta$
Projection(R)	0	$ R $
NestedLoopJoin(L, R, θ)	$P(L) \cdot (1 + P(R))$	$ L \cdot R \cdot c_\theta$
NestedLoopCrossJoin(L, R)	$P(L) \cdot (1 + P(R))$	$ L \cdot R $
HashJoin(L, R, θ)	$P(L) + P(R)$	$ L + R \cdot (1 + c_\theta)$
MergeJoin(L, R)	$P(L) + P(R)$	$ L + R $
Sort(R)	$2P(R) \cdot \lceil \log_M P(R) \rceil$	$ R \cdot \log_2 R $

Для операторов Filter и Projection стоимость ввода-вывода равна 0, потому что это конвейерные операторы, которые не используют диск.

1.8 Дифференциальный анализ физических планов

Множество правил трансформации и реализации, используемых в оптимизаторах запросов, изучено довольно хорошо и описано в академической ли-

тературе. Вместе с тем условия активации правил существенно различаются от системы к системе. Многие реализации оптимизаторов упускают возможности для оптимизации из-за слишком строгих условий активации правил или отсутствия определенных правил в их наборе.

Данная проблема особенно актуальна при разработке нового оптимизатора: после реализации базового набора правил возникает вопрос о полноте и корректности этого набора по сравнению с устоявшимися и проверенными временем промышленными системами. Такие системы аккумулировали обратную связь от пользователей и уточняли условия активации своих правил в течение многих лет. Не имея такого преимущества сложно добиться отличного качества построенных планов.

Решением может быть поиск несоответствий между поведением реализуемой системы и внешней эталонной системы путем применения дифференциального анализа планов.

Первым шагом нужно сформировать несколько наборов данных, состоящих из схем таблиц и их наполнения. Разные данные позволяют всесторонне исследовать правила активации, потому что они зависят от кардинальности отношений, наличия индексов и т.п.

Далее, по заданным схемам генерируются случайные SQL-запросы различной структуры и сложности. Такой подход исключает тривиальные планы и приближает условия работы оптимизатора к реальным нагрузкам при дальнейшем построении планов запросов.

Следующим шагом сгенерированные запросы транпилируются в диалекты SQL целевых промышленных СУБД. В качестве эталона рассматривается Microsoft SQL Server. Эта система является зрелой и имеет машиночитаемый формат вывода планов. Затем для получения физических планов выполняется аналог `EXPLAIN ANALYZE`.

Полученные физические планы промышленных СУБД конвертируются в формат сериализованного плана разрабатываемого оптимизатора. Современные СУБД предоставляют вывод планов в виде XML или JSON, что позволяет легко реализовать их разбор. Для конвертации используется отображение между физическими операторами разрабатываемой системы и операторами эталон-

ной СУБД. Например, Hash Match в SQL Server отображается в HashJoin в разрабатываемой системе, Clustered Index Scan — в IndexScan и так далее. Восстановление предикатов, ключей соединения и выражений проекции выполняется путем разбора соответствующих полей плана, причем не нужно поддерживать весь синтаксис предикатов, так как генератор запросов и схемы данных ограничивают множество допустимых выражений в планах.

Корректность конвертации выполняется по построению, но дополнительно проверяется путем исполнения исходного плана на внешней системе и сконвертированного плана на разрабатываемой системе с последующим сравнением полученных результатов при фиксированной схеме и различных вариантах сгенерированных для нее данных.

Конвертированные планы сравниваются с планами, порождаемыми разрабатываемым оптимизатором на исходных запросах, с целью определить, мог ли существующий набор правил разрабатываемого оптимизатора породить план, эквивалентный плану промышленной системы. Стоимостные модели и реализации операторов в разных СУБД различаются, поэтому план, оптимальный для эталонной системы, не обязательно будет оптимальным для разрабатываемой. По этой причине сравнение направлено на исследование полноты пространства поиска разрабатываемого оптимизатора, и не предполагает сравнение эффективности планов.

Для корректности структурного сравнения необходимо полное исследование пространства эквивалентных планов. Определение требуемых изменений лежит на разработчике системы, поэтому найденный запрос и примеры планов сохраняются для последующего анализа. В дальнейшем, полученные после конвертации “эталонные” планы можно использовать как тесты для оптимизатора, которые будут успешно выполняться после уточнения соответствующих правил активации.

В случае невозможности построить “эталонный” план разрабатываемым оптимизатором, предлагается искать ближайший план, т.е. план из пространства поиска разрабатываемого оптимизатора, максимально близкий к плану промышленной системы. При этом метрика расстояния между планами может быть определена как расстояние редактирования деревьев.

Анализ ближайшего плана может помочь упростить выявление трансформаций, которые разрабатываемый план по какой-то причине не применил.

Предлагаемый метод дифференциального анализа применим для сравнения как с проприетарными системами, так и с СУБД с открытым исходным кодом, позволяя уточнять условия применения правил.

Систематическое применение этого подхода позволяет построить итеративный процесс совершенствования оптимизатора:

- генерация запросов;
- запуск и сбор планов;
- приведение к единому виду, сравнение и выявление расхождений;
- добавление или корректировка правил;
- повторное сравнение.

Таким образом, дифференциальный анализ обеспечивает управляемое и измеримое улучшение качества оптимизатора и позволяет новым или открытым оптимизаторам быстрее достигать качества устоявшихся промышленных оптимизаторов.

2 Конструкторская часть

В рамках данной работы был разработан оптимизатор запросов для модельной реляционной СУБД.

Структурно реализация модельной СУБД состоит из следующих модулей:

- модуль синтаксического анализа SQL-запросов;
- модуль построения логического представления запроса;
- модуль стоимостной оптимизации на основе структуры Мемо;
- подсистема физического хранения на основе статических CSV-таблиц;
- модуль исполнения физических операторов.

Основой для проектирования оптимизатора была выбрана архитектура Cascades. В ней пространство допустимых планов формируется с помощью набора правил преобразования и хранится в структуре Мемо. Эта структура объединяет логически эквивалентные выражения в группы, что позволяет избежать повторного рассмотрения одинаковых поддеревьев. Поиск оптимального физического плана выполняется с учетом стоимостной модели и требуемых свойств результата, например порядка сортировки.

Данная архитектура была выбрана по причине ее модульности и расширяемости. Добавление новых логических преобразований и физических операторов выполняется путем реализации отдельных правил и не требует изменения основного алгоритма поиска. Это также удобно для итеративного процесса разработки, предполагающего постепенное улучшение качества итоговых планов путем уточнения и расширения пространства поиска.

2.1 Модули синтаксического анализа и построения логического представления

Модуль синтаксического анализа SQL-запросов отвечает за лексический и синтаксический разбор поступающих на вход SQL-запросов. В качестве основы для грамматики выбран диалект PostgreSQL [13], так как он широко используется на практике и имеет открытый исходный код. Использование готовой грамматики из официального репозитория [14] гарантирует совместимость.

В рамках работы поддерживается подмножество диалекта PostgreSQL, достаточное для исследования оптимизации запросов на чтение данных. В него входят операторы SELECT с предложениями FROM, WHERE, GROUP BY и ORDER BY, выбор всех столбцов или списка выражений, псевдонимы, соединения JOIN, INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN с условием ON, а также CROSS JOIN. Для группировки поддерживаются обычные выражения в GROUP BY и агрегатные функции SUM, COUNT и COUNT(*). Сортировка результата задается через ORDER BY по ссылкам на столбцы в прямом или обратном порядке.

Поддерживаемые скалярные выражения включают ссылки на столбцы, целочисленные константы, строковые литералы в одинарных кавычках, значения NULL, TRUE, FALSE и UNKNOWN, арифметические операции сложения, вычитания, умножения, деления, остатка от деления и возведения в степень, а также унарный минус. В предикатах поддерживаются операции сравнения, логические связки AND, OR и NOT, проверки IS NULL, IS TRUE, IS FALSE, IS UNKNOWN, условия BETWEEN и IN со списком значений. Такое подмножество покрывает основные конструкции, влияющие на построение логического плана: проекцию, фильтрацию, соединение, агрегацию и требование сортировки результата.

Модуль построения логического представления запроса отвечает за преобразование результата синтаксического анализа во внутреннее представление, пригодное для оптимизации. В качестве такого представления решено использовать дерево операторов реляционной алгебры. Этот подход является общепринятым и позволяет отделить особенности синтаксиса SQL от последующих этапов обработки запроса, а также выполнять оптимизацию над ограниченным набором формализованных операций: фильтрацией, проекцией, соединением, агрегацией и сортировкой.

2.2 Модули физического хранения и исполнения

Подсистема физического хранения отвечает за предоставление модулю исполнения табличных данных. Для хранения выбраны статические таблицы в формате CSV. Использование простого текстового формата позволяет сосредоточиться на проектировании оптимизатора и не усложнять прототип реализацией транзакций, индексов, буферного кэша и управления дисковыми страницами.

Кроме того, CSV-файлы удобно создавать, изменять и использовать при проведении тестов.

Данные хранятся в одной директории, где каждому отношению соответствует файл с CSV-таблицей. Название файла совпадает с названием отношения. Первая строка CSV-файла содержит описание атрибутов и их типов, последующие строки содержат кортежи отношения.

Дополнительно предусмотрена поддержка индексов. Индекс представляет собой отсортированный бинарный файл, содержащий целочисленные ключи и смещения соответствующих строк в CSV-файле. Использование смещений позволяет извлекать подходящие кортежи без полного сканирования отношения.

Модуль исполнения физических операторов отвечает за выполнение выбранного оптимизатором плана и формирование результата запроса. Физический план решено представлять в виде дерева операторов с единым интерфейсом взаимодействия. Каждый оператор получает данные от дочерних узлов, выполняет определенное преобразование и передает результат вышестоящему оператору. Такой подход обеспечивает модульность подсистемы исполнения и позволяет добавлять новые физические операторы независимо от уже реализованных.

2.3 Модуль стоимостной оптимизации планов

Модуль стоимостной оптимизации планов принимает на вход логическое представление запроса и строит физический план исполнения с минимальной оцененной стоимостью. В качестве основы выбран подход Cascades. Исходное дерево логических операторов помещается в структуру Мемо, после чего оптимизатор постепенно расширяет пространство поиска с помощью правил преобразования и правил реализации. Такой подход позволяет не хранить каждое дерево плана отдельно, а компактно представлять множество эквивалентных вариантов через группы Мемо.

Каждая группа Мемо соответствует множеству логически эквивалентных выражений, то есть выражений, возвращающих один и тот же результат. Логические выражения содержат ссылки на дочерние группы, поэтому один узел

может представлять сразу несколько вариантов построения поддерева. В процессе оптимизации в группы добавляются как новые логические выражения, полученные правилами трансформации, так и физические выражения, полученные правилами реализации.

Правило трансформации создает новое логическое выражение и добавляет его в ту же группу Мемо, что и исходное выражение. Следовательно, правило должно сохранять семантику запроса. Правило реализации создает физическое выражение, которое может быть передано модулю исполнения.

Реализованы следующие трансформационные правила:

- коммутативность соединения;
- ассоциативность соединения;
- разбиение конъюнкции на последовательность фильтров;
- объединение последовательных фильтров;
- проталкивание фильтра через проекцию;
- проталкивание фильтра во входы соединения;
- преобразование выражения IN в цепочку сравнений.

И следующие правила реализации:

- последовательное сканирование таблицы;
- фильтрация;
- проекция;
- хэш-агрегация;
- потоковая агрегация;
- декартово произведение вложенными циклами;
- соединение вложенными циклами;
- хэш-соединение;
- соединение слиянием.

Данный набор правил все основные алгоритмы, реализующие операторы реляционной алгебры, и включает самые важные правила трансформации. Также более полное покрытие правилами реализации хорошо подходит для демонстрации возможностей дифференциального сравнения планов для последующего расширения покрытия правилами трансформации.

Выбор лучшего плана выполняется на основе стоимостной модели. Стоимость физического оператора рассчитывается с учетом оценок кардинальности входных и выходных групп. Формулы для вычисления стоимости откалиброваны по результатам бенчмарков и представлены в таблице 5. Кардинальность базовых отношений берется из описания входных таблиц, а для фильтров, соединений, агрегаций и проекций оценивается эвристически. Чтобы сократить перебор, оптимизатор использует метод ветвей и границ. Если локальная стоимость или нижняя оценка стоимости поддерева уже превышает текущий лучший результат, соответствующая ветвь пространства поиска не рассматривается дальше.

Для оценки кардинальности промежуточных результатов используется коэффициент селективности $sel(p)$, показывающий долю кортежей, удовлетворяющих предикату p . Используемые формулы приведены в таблице 3.

Такие формулы выбраны как простая эвристическая модель, не требующая гистограмм, сведений о количестве различных значений и статистик корреляции между атрибутами. Равенство атрибута константе считается достаточно селективным, поэтому ему сопоставляется коэффициент 0,10. Диапазонные и неизвестные условия оцениваются нейтральным коэффициентом 0,50, так как без статистики нельзя надежно определить, какую часть отношения они оставят. Для IN используется сумма селективностей отдельных равенств с верхней границей 0,50, чтобы длинный список значений не делал оценку слишком оптимистичной.

Формулы для AND, OR и NOT основаны на предположении независимости предикатов. Для эквисоединения двух атрибутов используется оценка $1 / \max(n_l, n_r)$, соответствующая типичному случаю соединения по ключу или внешнему ключу, размер результата в таком случае оказывается сопоставим с размером большего входа, а не с полным декартовым произведением. Для неизвестных условий соединения применяется селективность 1, чтобы не занижать стоимость плана при отсутствии надежной информации.

Отдельно учитываются физические свойства результата. В данной работе таким свойством является порядок строк, требуемый предложением ORDER BY. Если выбранный план не обеспечивает требуемую сортировку, оптимизатор

может добавить обеспечивающий физический оператор сортировки и сравнить его стоимость с другими альтернативами. Результатом работы модуля является дерево физических операторов, которое передается в модуль исполнения.

3 Технологическая часть

Программа написана на языке C++ с использованием стандарта C++23. Этот язык популярен в сфере разработки СУБД, являясь достаточно низкоуровневым и производительным, но в то же время удобным для реализации больших систем. Для сборки применяется CMake и его встроенные средства для зависимостей в виде библиотек на C++. Окружение для разработки описывается декларативно и воспроизводимо при помощи функционального доменно-специфичного языка `nix` для соответствующего пакетного менеджера.

Для синтаксического анализа выбран генератор парсеров ANTLR4, ввиду простоты и удобства в использовании. Асинхронное взаимодействие физических операторов построено на Boost.Asio, по причине совместимости со встроенными в язык корутинами и понятной модели асинхронности. Для тестирования корректности и производительности используется Google Tests, Google Benchmarks и pytest.

3.1 Модуль синтаксического анализа

Для реализации парсера был выбран ANTLR4, ввиду его удобства и возможности генерировать лексические и синтаксические анализаторы под практически любой язык программирования.

Грамматика PostgreSQL была взята из официального репозитория проекта ANTLR4 [15]. Она была портирована автоматически из грамматики для Bison [14] из официального репозитория Postgres, поэтому является наиболее полной из существующих.

ANTLR4 генерирует несколько файлов на C++, в том числе лексический и синтаксический анализаторы, а также класс `Visitor` для обхода дерева разбора. Для интересных конструкций SQL переопределены методы посещения узлов грамматики. Например, предложение `WHERE` преобразуется в логический оператор фильтрации, список выражений после `SELECT` преобразуется в проекцию, а список таблиц после `FROM` — в дерево соединений.

Результатом работы парсера является структура `ParsedQuery`. Она содержит логический оператор верхнего уровня и необязательное требование к

порядку строк для ORDER BY. Логические операторы и скалярные выражения представлены алгебраическими типами на основе `std::variant`. В них входят операторы таблицы, фильтрации, проекции, агрегации, соединения, ссылки на атрибуты, константы и арифметические выражения (листинг 5).

Листинг 5: Объявление типов для операторов реляционной алгебры и выражений.

```
using Operator = std::variant<Table, Projection, Filter, Aggregation,  
    ↪ CrossJoin, Join>;  
  
using Expression = std::variant<BinaryExpression, Attribute, IntConst,  
    ↪ StringConst, UnaryExpression, InExpression, AggregateExpression,  
    ↪ Literal>;  
  
struct Projection {  
    std::vector<Expression> expressions;  
    std::shared_ptr<Operator> source;  
    std::vector<std::optional<std::string>> aliases;  
  
    bool operator==(const Projection& other) const;  
};
```

Выражения хранятся в алгебраическом типе `Expression`, реализованном аналогично `Operator`.

Функция `GetAST` внутри строит абстрактное синтаксическое дерево и обходит его с помощью `Visitor`, который и генерирует представление запроса в виде дерева из `Operator`.

Дополнительно реализована визуализация дерева из операторов реляционной алгебры в формате `Graphviz` с помощью функции `GetDotRepresentation`.

3.2 Модуль оптимизации запросов

Модуль оптимизации запросов отвечает за преобразование логического дерева операторов в физический план исполнения с минимальной оцененной стоимостью. Пространство исследуемых эквивалентных планов эффективно хранится в структуре `Мето` и расширяется с помощью добавления новых пра-

вил. Поиск выполняется сверху вниз с использованием стоимостной модели, физических свойств и метода ветвей и границ.

Центральное место в оптимизаторе занимает структура Мемо, которая хранит группы логически эквивалентных выражений. Выражения из одной группы возвращают одинаковый результат. Например, $A \bowtie B$ и $B \bowtie A$ являются эквивалентными.

Дочерние узлы выражения ссылаются не на конкретные операторы, а на группы. Благодаря этому одно выражение компактно представляет множество деревьев. Объявления классов Мемо и Group приведены в листинге 6.

Листинг 6: Основные элементы структуры Мемо.

```
class Memo {
public:
    utils::NotNull<LogicalExpr*> AddGroup(LogicalOperator op);
    utils::NotNull<LogicalExpr*> AddLogicalExprToGroup(
        utils::NotNull<Group*> group, LogicalOperator op);
    utils::NotNull<LogicalExpr*> Populate(const Operator& op);

private:
    std::deque<Group> groups_;
    std::unordered_map<std::string, LogicalExpr*> expr_index_;
};

class Group {
public:
    utils::NotNull<LogicalExpr*> AddLogicalExpr(LogicalOperator op);
    utils::NotNull<PhysicalExpr*> AddPhysicalExpr(
        PhysicalOperator op, bool is_enforcer = false);

    LogicalExprs GetLogicalExprs();
    PhysicalExprs GetPhysicalExprs();
};
```

Метод Мемо::Populate рекурсивно обходит исходное логическое дерево и создает первоначальные группы. Для предотвращения повторного добавления выражений используется отображение expr_index_. Ключ строится из типа оператора, его параметров и идентификаторов дочерних групп. Таким образом, одинаковые выражения добавляются в Мемо только один раз.

Каждая группа содержит логические и физические выражения. Логические выражения описывают семантику запроса, а физические выражения опре-

деляют конкретный способ исполнения. Например, логическому соединению могут соответствовать физические операторы соединения вложенными циклами и хеш-соединения.

3.2.1 Правила трансформации и реализации

Расширение пространства поиска выполняется с помощью правил. Реализовано два типа правил: трансформационные и реализационные. Правила реализуют единый интерфейс, состоящий из условия применимости и метода для применения правила. Условие применимости выполняет быструю структурную проверку по дереву выражений в Мемо. Метод применения правила порождает необходимые новые группы и результирующее логическое или физическое выражение. В качестве примера в листинге 17 приведена реализация правила коммутативности соединений.

Для каждого выражения хранится информация о ранее примененных правилах. Это предотвращает повторное выполнение одного правила над одним выражением и ограничивает образование циклов при заполнении Мемо.

3.2.2 Алгоритм поиска

Стоимость физического плана зависит от количества обрабатываемых кортежей. Для получения этой величины используется класс `CardinalityEstimates`. Оценка кардинальности вычисляется для группы Мемо и кэшируется.

Стоимость плана вычисляется как сумма локальных стоимостей физических операторов. Фрагмент реализации стоимостной модели приведен в листинге 16.

Класс `Optimizer` реализует алгоритм поиска. При создании объекта исходное логическое дерево помещается в Мемо. Поиск запускается для корневой группы и требуемых физических свойств результата.

Алгоритм использует стек отложенных задач. Это позволяет разделить процесс на небольшие операции: исследование группы, применение правила, оптимизацию физического выражения и обработку его дочерних узлов. Основной метод оптимизации группы приведен в листинге 18.

Если группа еще не исследована, в стек добавляются две задачи. Первая расширяет группу с помощью правил, вторая повторно запускает ее оптимизацию после расширения. Если группа уже исследована, для каждого физического выражения вычисляется полная стоимость с учетом дочерних групп.

Для каждой пары из группы и требуемого набора свойств хранится лучший найденный вариант. Ключ такого варианта представлен структурой WinnerKey (листинг 7). Хранение победителя отдельно для каждого набора свойств необходимо, поскольку самый дешевый неотсортированный план может отличаться от самого дешевого плана, удовлетворяющего требованию ORDER BY.

Листинг 7: Ключ для хранения лучшего плана по группе и свойствам.

```
struct WinnerKey {  
    Group* group;  
    PropertySet required;  
};
```

Дочерние группы физического выражения оптимизируются последовательно. После получения победителя дочерней группы его стоимость прибавляется к накопленной стоимости. Если все дочерние группы обработаны, проверяются выходные свойства плана. Затем найденный вариант может стать новым лучшим планом.

3.2.3 Метод ветвей и границ

Полный перебор пространства планов быстро становится непрактичным при увеличении количества соединяемых таблиц. Для сокращения перебора используется метод ветвей и границ.

Для каждой группы вычисляется нижняя граница стоимости. Она равна минимальной сумме нижней оценки локального оператора и нижних оценок дочерних групп. Например, для логического соединения выбирается минимум между стоимостью хеш-соединения и стоимостью соединения вложенными циклами:

$$LB_{\text{join}} = \min(69(N_l + N_r), 70N_lN_r).$$

Если нижняя граница не меньше стоимости уже известного решения, исследование ветви прекращается. Фрагмент реализации приведен в листинге 8.

Листинг 8: Вычисление нижней границы стоимости группы.

```
int64_t Optimizer::LowerBoundCost(Group* group) {
    if (auto it = lower_bounds_.find(group);
        it != lower_bounds_.end()) {
        return it->second;
    }
    int64_t best = std::numeric_limits<int64_t>::max();
    for (auto expr : group->GetLogicalExprs()) {
        int64_t cost = LowerBoundLocalCost(expr, cardinality_);
        for (auto child : GetChildren(expr)) {
            cost += LowerBoundCost(child);
        }
        best = std::min(best, cost);
    }
    lower_bounds_[group] = best;
    return best;
}
```

Метод `OptimizeInputs` также уменьшает границу для очередного дочернего узла. Из общей допустимой стоимости вычитаются уже накопленная стоимость и нижние оценки еще не обработанных дочерних групп. Благодаря этому заведомо неоптимальные ветви отбрасываются до построения полного плана.

3.2.4 Физические свойства

Некоторые требования к результату нельзя выразить только стоимостью. Например, конструкция `ORDER BY` требует от результата определенного порядка кортежей. Для представления таких требований используется класс `PropertySet`.

Например, порядок сортировки считается удовлетворяющим требованию, если требуемые ключи являются префиксом фактически полученного порядка. Таким образом сортировка по (a, b) удовлетворяет требованию сортировки по a .

Некоторые операторы сохраняют порядок дочернего узла. Фильтрация и проекция сохраняют свойства входа. Соединение вложенными циклами со-

храняет свойства левого входа. Хеш-соединение и агрегация не гарантируют сохранение порядка.

Если подходящий план не найден, оптимизатор пытается добавить обеспечивающий оператор. Для сортировки используется класс `SortEnforcer`. Он проверяет наличие требуемых атрибутов в схеме группы и создает физический оператор сортировки.

Листинг 9: Добавление оператора сортировки для обеспечения свойства.

```
std::optional<PhysicalOperator> SortEnforcer::TryBuild(
    utils::NotNull<Group*> group,
    const PropertySet& required,
    SchemaCatalog& schema) const {
    const auto* sort = required.Get<SortProperty>();
    if (!sort) return std::nullopt;
    if (!ContainsRequiredKeys(schema.GetSchema(group),
                               sort->order)) {
        return std::nullopt;
    }
    return physical::Sort{group, sort->order};
}
```

3.2.5 Формирование физического плана

После завершения поиска метод `BuildOptimalPlan` рекурсивно восстанавливает итоговое дерево физических операторов. Для каждой группы выбирается сохраненный лучший план с учетом требуемых свойств. Затем аналогичным образом восстанавливаются планы дочерних групп.

Результатом работы оптимизатора является объект `PhysicalPlanNode`, который не зависит от структуры Мемо. Он может быть сериализован для анализа или передан модулю исполнения запросов.

Таким образом, модуль оптимизации разделяет представление пространства планов, правила его расширения, стоимостную модель и алгоритм поиска. Такая организация позволяет добавлять новые правила и физические операторы без изменения основной логики оптимизатора.

3.3 Модуль физического хранения данных

Для чтения таблиц реализован класс `CsvDirSequentialScanner`. Он выполняет последовательное сканирование CSV-файла и передает прочитанные кортежи модулю исполнения физических операторов.

Список доступных индексов задается в файле `indexes.meta`, расположенном в директории с данными. Каждая строка файла содержит название отношения, название индексируемого атрибута, тип индекса и имя бинарного файла (листинг 10).

Листинг 10: Пример файла с метаданными о доступных индексах.

```
users id sorted users.id.sorted.idx
```

Для чтения данных с использованием индекса реализован класс `CsvDirIndexedScanner`. Если указанный в метаданных бинарный файл отсутствует, индекс строится при первом обращении к таблице. Для поиска диапазона подходящих ключей используется упорядоченность записей индекса.

Поддерживаются индексы только для целочисленных атрибутов. Индексное сканирование применяется для предикатов сравнения атрибута с целочисленной константой. Из возможных операций доступно равенство, строгое и нестрогое сравнение. После чтения найденных строк дополнительно вычисляется исходный предикат, что обеспечивает корректную обработку составных условий.

3.4 Модуль исполнения физических планов

За исполнение запросов отвечает класс `Executor` (листинг 11). Он получает физический план, запускает исполнение его операторов и формирует результирующее отношение. Физический план представлен деревом объектов типа `PhysicalPlanNode`. Для каждого типа узла предусмотрена отдельная функция исполнения реализующая единый интерфейс и исполняющаяся независимо.

Листинг 11: Основные элементы объявления класса Executor.

```
template <typename ExpressionExecutor = InterpretedExpressionExecutor>
class Executor {
public:
    Executor(SequentialScan seq_scan,
             IndexScan index_scan,
             boost::asio::any_io_executor executor);
    boost::asio::awaitable<Result<Relation>>
    Execute(const PhysicalPlanNode& op);
private:
    SequentialScan sequential_scan_;
    IndexScan index_scan_;
    ExpressionExecutor expression_executor_;
};
```

Класс параметризуется интерфейсами SequentialScan, IndexScan и ExpressionExecutor. Первые два интерфейса отделяют исполнение физического плана от конкретного способа хранения данных. SequentialScan выполняет полное чтение таблицы, а IndexScan получает дополнительный предикат и извлекает строки с использованием индекса. Благодаря этому исполнитель не зависит от формата CSV-файлов и устройства индексных структур.

Параметр ExpressionExecutor определяет способ вычисления скалярных выражений. Реализованы стратегии для интерпретации выражений и для JIT-компиляции. По умолчанию используется интерпретируемый вариант.

Значения атрибутов представлены структурой Value. Она содержит признак NULL и значение одного из поддерживаемых типов: целого числа, логического значения или идентификатора строки. Исходное значение для строк сохраняется в пул, а в кортеже хранится числовой идентификатор строки. Кортеж представлен вектором значений, а отношение состоит из описания атрибутов и списка кортежей.

Коммуникация между физическими операторами осуществляется с помощью асинхронных каналов. Используются два типа каналов: AttributesInfoChannel для однократной передачи схемы отношения и TuplesChannel для передачи данных. Кортежи передаются векторами. Это уменьшает число операций синхронизации и позволяет потоковым операторам начинать обработку до полного завершения дочерних узлов плана.

Для реализации каналов используется `boost::asio::experimental::concurrent_channel`. Данный класс обеспечивает потокобезопасность, буферизацию, асинхронную передачу данных и совместимость с корутинами. Такой подход позволяет единообразно организовать взаимодействие между операторами независимо от конкретных алгоритмов для их реализации.

3.5 Руководство пользователя

Для сборки программы требуются CMake, компилятор C++ с поддержкой C++23, LLVM, ANTLR4 и библиотеки Boost. Дополнительно требуется Python, pytest и Docker. В репозитории присутствует файл `flake.nix`, описывающий необходимые зависимости, поэтому в системе с пакетным менеджером Nix рекомендуется открыть окружение при помощи команды `nix develop`.

Все команды далее выполняются из корневого каталога проекта.

Для выполнения конфигурации и сборки следует запустить команды на листинге 12.

Листинг 12: Конфигурация и сборка проекта.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -- -j 6
```

После успешной сборки исполняемый файл находится по пути `build/bin/sql`. Для повторной сборки также можно использовать цель `make build`.

Пользователь должен создать отдельную директорию и поместить в нее CSV-файлы. Имя каждого файла без расширения считается именем таблицы. В заголовке каждого файла для столбца указывается имя и тип через двоеточие. Поддерживаются типы `int` и `string`. Пустое SQL-значение записывается как `NULL`. Для первого запуска можно использовать готовые тестовые данные в `test/static/executor/test_data`. SQL-запрос передается программе через стандартный поток ввода. Простейший пример запуска представлен на листинге 13 и на рисунке 11.

Листинг 13: Примеры команд для запуска программы.

```
echo 'SELECT users.id FROM users;' | \  
./build/bin/sql --data-dir test/static/executor/test_data  
  
echo 'SELECT users.id FROM users WHERE users.age > 18 \  
ORDER BY users.id;' | \  
./build/bin/sql --data-dir test/static/executor/test_data \  
--print-ast --print-plan
```

Программа выводит заголовок отношения, содержащий имена и типы столбцов, а затем полученные строки. Если запрос не содержит ORDER BY, строки вывода сортируются лексикографически для воспроизводимости результата.

Для диагностики предусмотрены параметры `-print-ast` и `-print-plan`. Первый параметр выводит логическое дерево после синтаксического анализа, второй — выбранный физический план. Пример использования представлен на листинге 13.

Если план недостижим, программа выводит сообщение NOT REACHABLE и описание первого обнаруженного расхождения. Параметр `--data-dir` необходим для загрузки схем таблиц. В частности, схема используется при проверке достижимости планов с сортировкой.

Для поиска ошибок исполнения реализован дифференциальный фаззер. Он генерирует случайные SQL-запросы, выполняет их в модельной СУБД и Microsoft SQL Server, после чего сравнивает полученные результаты. Перед запуском необходимо поднять контейнер с Microsoft SQL Server. Фаззер запускается из корневого каталога проекта командой на листинге 14.

Второй фаззер проверяет полноту набора правил оптимизатора. Он получает физический план запроса из Microsoft SQL Server, преобразует его во внутренний формат проекта и запускает программу в режиме `--check-reachable`, как показано на листинге 14.

Листинг 14: Запуск фаззеров.

```
docker compose up -d
python -m research.fuzz.reach_fuzz \
  --cli build/bin/sql \
  --data-dir test/static/executor/test_data
python -m research.fuzz.diff_fuzz \
  --cli build/bin/sql \
  --data-dir test/static/executor/test_data
```

4 Аналитическая часть

Тестирование разработанной модельной СУБД выполнялось на нескольких уровнях. Для проверки отдельных компонентов использовались модульные тесты. Корректность выполнения запросов и полнота пространства поиска оптимизатора дополнительно проверялись с помощью фаззинга. Для оценки производительности и калибровки стоимостной модели были реализованы бенчмарки. Вспомогательные инструменты на языке Python проверяются с помощью библиотеки `pytest`.

Модульные тесты реализованы с использованием библиотеки `GoogleTest`. Они проверяют синтаксический анализ SQL-запросов, исполнение физических операторов, индексное сканирование, вычисление выражений, сериализацию физических планов, структуру Мемо, правила оптимизации и обработку свойств сортировки. Пример теста синтаксического анализатора приведен в листинге 15. Также проверяется выбор физического плана. Например, при различных оценках кардинальности оптимизатор должен выбирать подходящий порядок соединения таблиц.

Листинг 15: Пример модульного теста.

```
TEST(ParserTest, SelectSingleColumnFromSingleTable) {
    std::stringstream s{"SELECT users.id FROM users;"};

    Operator got = GetAST(s).value().op;

    ASSERT_THAT(got, VariantWith<Projection>(
        Projection{
            std::vector<Expression>{{Attribute{"users", "id"}}},
            std::make_shared<Operator>(Table{"users"})
        }));
}
```

Для проверки вспомогательных инструментов используются тесты на языке Python. Они разделены на две группы. Первая группа проверяет преобразование физических планов Microsoft SQL Server из формата `ShowPlanXML` во внутренний текстовый формат проекта. Рассматриваются последовательное и индексное сканирование, фильтрация, сортировка, агрегация и соединения.

Часть тестов использует заранее подготовленные XML-фрагменты, часть может быть запущена с подключением к экземпляру Microsoft SQL Server.

Вторая группа относится к стандартному Star Schema Benchmark. Она проверяет преобразование исходных таблиц SSB в CSV-формат модельной СУБД, обработку некорректных входных данных и успешное выполнение стандартных аналитических запросов.

4.1 Дифференциальный анализ и тестирование производительности

Для проверки системы на большом количестве входных данных реализован генератор случайных SQL-запросов. Он строит запросы из поддерживаемого подмножества языка: проекции, фильтры, соединения, группировки, агрегатные функции, сортировки, строковые выражения, IN и BETWEEN.

Дифференциальный фаззер выполняет каждый сгенерированный запрос в модельной СУБД и в Microsoft SQL Server. Результаты сравниваются как мультимножества строк. Для запросов с ORDER BY дополнительно проверяется корректность порядка результатов.

Отдельный фаззер используется для исследования полноты набора правил оптимизации. Для случайного запроса из Microsoft SQL Server извлекается физический план, после чего план преобразуется во внутренний формат проекта. Затем модельный оптимизатор выполняет полный перебор пространства поиска и проверяет, достижим ли полученный план с помощью реализованных правил. Такой подход позволяет находить отсутствующие преобразования и физические операторы.

Тесты производительности реализованы с помощью библиотеки Google Benchmark. Для запросов предусмотрены два режима построения физического плана: Naive и Optimized. В первом случае физический план строится непосредственно по логическому дереву без применения оптимизатора. Во втором случае используется стоимостная оптимизация. Сравнение этих режимов позволяет оценить влияние выбранного физического плана на время выполнения запроса.

Бенчмарки выполняются как для небольших синтетических тестовых запросов, так и для набора аналитических запросов Star Schema

Benchmark. Дополнительно поддерживается сравнение интерпретируемого и JIT-компилируемого способов вычисления выражений.

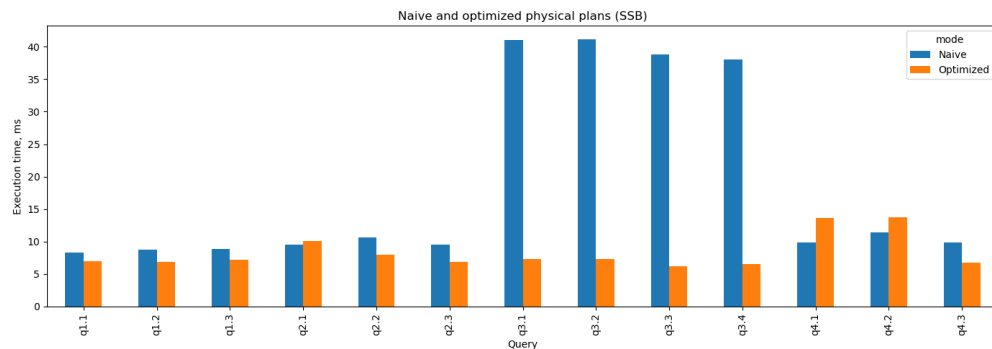


Рисунок 5 – Сравнение времени выполнения запросов при наивном и оптимизированном построении физического плана.

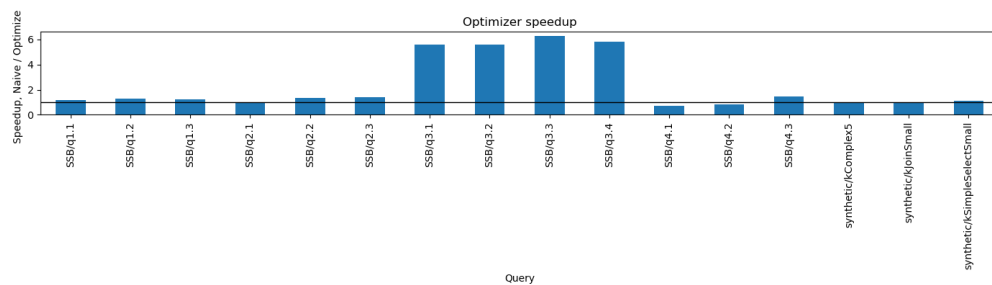


Рисунок 6 – Относительное ускорение выполнения запросов после применения оптимизатора.

В таблице 2 приведено среднее время выполнения трех повторений запросов SSB. Ускорение рассчитывается как отношение времени выполнения наивного плана ко времени выполнения оптимизированного плана.

Видно, что на стандартном наборе запросов оптимизатор дает на 20 – 50% раз, а на некоторых запросах в 6 и более раз.

Таблица 2 – Результаты выполнения запросов Star Schema Benchmark.

Запрос	Naive, мс	Optimized, мс	Ускорение
q1.1	8.11	6.51	1.25
q1.2	8.20	6.59	1.24
q1.3	8.00	6.58	1.22
q2.1	8.80	9.33	0.94
q2.2	8.84	6.18	1.43
q2.3	8.68	5.80	1.50
q3.1	35.67	5.90	6.04
q3.2	35.88	5.58	6.43
q3.3	38.05	5.86	6.49
q3.4	37.12	5.77	6.43
q4.1	9.06	12.21	0.74
q4.2	9.05	11.12	0.81
q4.3	9.28	6.31	1.47

4.2 Калибровка стоимостной модели

Для калибровки коэффициентов в стоимостных формулах реализованы отдельные бенчмарки физических операторов. Измеряется время выполнения последовательного сканирования, фильтрации, проекции, сортировки, агрегации, хеш-соединения, соединения вложенными циклами и декартова произведения (рисунок 7).

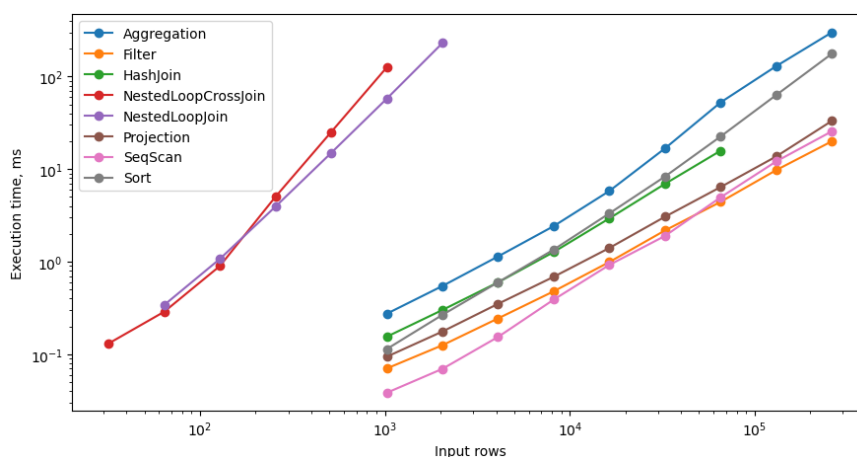


Рисунок 7 – Зависимость времени выполнения физических операторов от размера входных данных.

При выполнении этих бенчмарков данные хранятся в оперативной памяти. Это позволяет уменьшить влияние файловой системы и измерять преимуще-

ственно стоимость самого физического оператора. Для каждого оператора выполняются замеры на таблицах различного размера.

Дополнительно формируются группы запусков с приблизительно одинаковой расчетной стоимостью. Для этого размер входных отношений выбирается на основе стоимостной формулы оператора. Сопоставление расчетной стоимости с фактическим временем выполнения позволяет сравнить операторы между собой и скорректировать коэффициенты модели (рисунок 10). Благодаря этому оптимизатор принимает решения на основе оценок, соответствующих характеристикам реализованного исполнителя.

Выведенные коэффициенты дополнительно валидируются на случайных запросах путем сравнения полной стоимости плана и времени его исполнения (рисунок 12).

ЗАКЛЮЧЕНИЕ

В ходе работы была изучена архитектура оптимизаторов Cascades и на ее основе спроектирован собственный оптимизатор для модельной реляционной СУБД.

Было формализовано поддерживаемое подмножество SQL и соответствующее ему представление в терминах реляционной алгебры. Реализованная система поддерживает запросы на чтение данных с основными операторами диалекта PostgreSQL.

Был реализован алгоритм поиска оптимального физического плана и структура Мемо для эффективного хранения групп логически эквивалентных выражений. В результате система выбирает физический план с минимальной оценочной стоимостью с учетом требуемых свойств результата.

Для формирования пространства поиска был разработан набор правил трансформации и реализации для основных операторов реляционной алгебры. Реализованные правила позволяют изменять порядок и группировку соединений, преобразовывать предикаты и выражения, а также сопоставлять логическим операторам физические реализации.

Для ускорения поиска был реализован метод ветвей и границ. Оптимизатор использует текущую лучшую стоимость и нижние оценки стоимости поддеревьев для отсекаания заведомо неоптимальных вариантов, что уменьшает объем исследуемого пространства планов ухудшения качества результирующих планов.

Для последующего развития оптимизатора был реализован метод дифференциального анализа физических планов. Подготовлены инструменты генерации запросов, сравнения результатов исполнения и сопоставления планов с промышленной СУБД Microsoft SQL Server.

Тем самым цель работы была достигнута. Получена работоспособная и расширяемая система, которую можно использовать как основу для дальнейшего уточнения правил оптимизации и увеличения пространства исследуемых планов исполнения запросов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Исследование рынка СУБД. — URL: <https://www.mordorintelligence.com/industry-reports/database-market> ; (дата обращения: 17.05.2026).
2. Petrov A. Database Internals: A deep dive into how distributed data systems work. — O'Reilly Media, 2019. — (дата обращения: 17.05.2026).
3. Database system concepts / A. Silberschatz, H. F. Korth, S. Sudarshan [и др.]. — McGraw-hill New York, 2020. — (дата обращения: 17.05.2026).
4. Access path selection in a relational database management system / P. G. Selinger [и др.] // Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data. — 1979. — С. 23—34. — (SIGMOD '79). — DOI: 10.1145/582095.582099. — (дата обращения: 01.05.2026).
5. Introduction to the Join Ordering Problem. — URL: <https://www.querifylabs.com/blog/introduction-to-the-join-ordering-problem> ; (дата обращения: 01.05.2026).
6. Graefe G., McKenna W. J. The Volcano Optimizer Generator: Extensibility and Efficient Search // Proceedings of IEEE 9th International Conference on Data Engineering. — 1993. — С. 209—218. — DOI: 10.1109/ICDE.1993.344061. — (дата обращения: 01.05.2026).
7. Graefe G. Volcano - an extensible and parallel query evaluation system // IEEE Transactions on Knowledge and Data Engineering. — 2002. — Т. 6, № 1. — С. 120—135. — (дата обращения: 17.05.2026).
8. Apache Calcite: A foundational framework for optimized query processing over heterogeneous data sources / E. Begoli [и др.] // Proceedings of the 2018 International Conference on Management of Data. — 2018. — С. 221—230. — (SIGMOD '18). — DOI: 10.1145/3183713.3190662.
9. Graefe G. The Cascades Framework for Query Optimization // IEEE Data Engineering Bulletin. — 1995. — Т. 18, № 3. — С. 19—29. — (дата обращения: 01.05.2026).

10. Xu Y. Efficiency in the Columbia Database Query Optimizer : дис. . . . канд. / Xu Yongwen. — Portland State University, 1998.
11. Ding B., Narasayya V., Chaudhuri S. Extensible Query Optimizers in Practice // Foundations and Trends in Databases. — 2024. — Т. 14, № 3/4. — С. 186—402. — DOI: 10.1561/19000000077. — (дата обращения: 01.05.2026).
12. How good are query optimizers, really? / V. Leis [и др.] // Proceedings of the VLDB Endowment. — 2015. — Т. 9, № 3. — С. 204—215. — (дата обращения: 01.05.2026).
13. Документация PostgreSQL. — URL: <https://www.postgresql.org/docs/current/sql-select.html> ; (дата обращения: 17.05.2026).
14. Грамматика PostgreSQL. — URL: <https://github.com/postgres/postgres/blob/master/src/backend/parser/gram.y> ; (дата обращения: 17.05.2026).
15. Грамматика PostgreSQL для ANTLR4. — URL: <https://github.com/antlr/grammars-v4/blob/master/sql/postgresql/PostgreSQLParser.g4> ; (дата обращения: 17.05.2026).

ПРИЛОЖЕНИЕ А

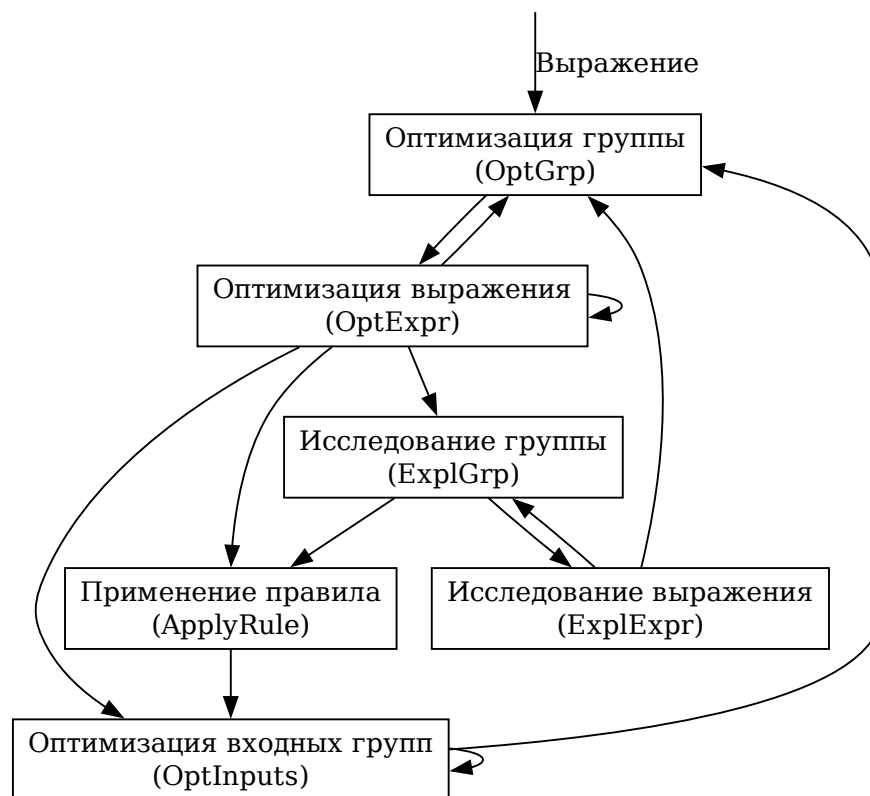


Рисунок 8 – Схема взаимодействия задач в оптимизаторе Cascades

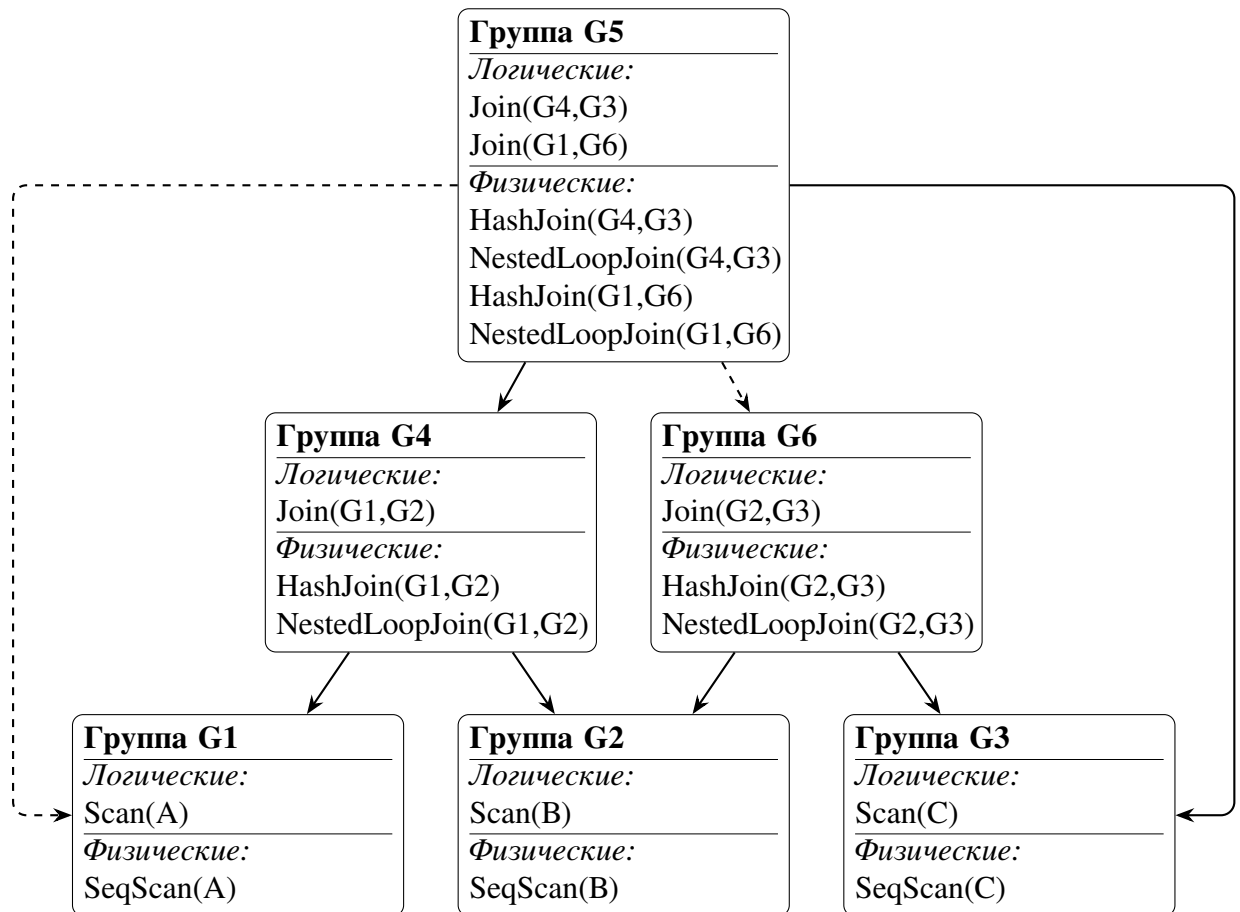


Рисунок 9 – Пример структуры Мемо.

Таблица 3 – Формулы оценки селективности предикатов.

Вид предиката	Формула селективности
$a = c$, где a — атрибут, c — константа	$sel(a = c) = 0,10$
$a < b$, $a > b$, $a \leq b$, $a \geq b$	$sel(p) = 0,50$
$p_1 \wedge p_2$	$sel(p_1 \wedge p_2) = sel(p_1) \cdot sel(p_2)$
$p_1 \vee p_2$	$sel(p_1 \vee p_2) = sel(p_1) + sel(p_2) - sel(p_1) \cdot sel(p_2)$
$\neg p$	$sel(\neg p) = 1 - sel(p)$
$a \in \{c_1, \dots, c_k\}$	$sel(p) = \min(0,50, 0,10k)$
$a \notin \{c_1, \dots, c_k\}$	$sel(p) = 1 - \min(0,50, 0,10k)$
$a = b$ в условии соединения, где a и b — атрибуты разных входов	$sel(a = b) = \frac{1}{\max(1, \max(n_l, n_r))}$
$\theta_1 \wedge \theta_2$ в условии соединения	$sel(\theta_1 \wedge \theta_2) = sel(\theta_1) \cdot sel(\theta_2)$
Иной или неизвестный предикат фильтрации	$sel(p) = 0,50$
Иной или неизвестный предикат соединения	$sel(\theta) = 1$

Листинг 1 Алгоритм поиска (часть 1).

```
1: procedure Optimize(root, required)
2:   Push task OptimizeGroup(root.group, required,  $+\infty$ )
3:   while task stack is not empty do
4:     Pop and execute top task
5:   return BuildPlan(root.group, required)
6: procedure ExploreGroup(group, limit)
7:   Mark group as explored
8:   for each logical expression expr  $\in$  group do
9:     Push task ExploreExpression(expr, limit)
10: procedure ExploreExpression(expr, limit)
11:   TryRules(expr, limit)
12:   if expr is already explored then
13:     return
14:   Mark expr as explored
15:   for each child  $\in$  Children(expr) do
16:     Register expr as parent of child
17:     if child is not explored then
18:       Push task ExploreGroup(child, limit)
19: procedure TryRules(expr, limit)
20:   for each applicable transformation rule r do
21:     Push task ApplyTransformation(r, expr, limit)
22:   for each applicable implementation rule r do
23:     Push task ApplyImplementation(r, expr)
```

Листинг 2 Алгоритм поиска (часть 2).

```
1: procedure OptimizeGroup(group, required, limit)
2:   if winner[group, required] exists or group is explored and
   LowerBound(group)  $\geq$  limit then
3:     return
4:   if group is not explored then
5:     Push task OptimizeGroup(group, required, limit)
6:     Push task ExploreGroup(group, limit)
7:     return
8:   for each physical expression phys  $\in$  group do
9:     if phys is not enforcer and local_cost[phys] < limit then
10:      Push task OptimizeInputs(phys, required,  $\emptyset$ , local_cost[phys], limit, 0)
11:   for each enforcer applicable to (group, required) do
12:     Add enforcer expression phys and compute local_cost[phys]
13:     if local_cost[phys] < limit then
14:       Push task OptimizeInputs(phys, required,  $\emptyset$ , local_cost[phys], limit, 0)
15:   procedure OptimizeInputs(phys, required, delivered, accum, limit, i)
16:     if winner[phys.group, required] exists then
17:       limit  $\leftarrow$  min(limit, winner[phys.group, required].cost)
18:     if accum  $\geq$  limit then
19:       return
20:     if i = |children| then
21:       props  $\leftarrow$  DeriveOutputProps(phys, delivered)
22:       Update winner and return
23:     Calc child limit and requirements
24:     Push task OptimizeGroup(child, child_required, child_limit)
```

Таблица 4 – Правила трансформации.

№	Название	Правило	Комментарий
1	Коммутативность соединения	$\frac{R \bowtie_{\theta} S}{\text{внутреннее или полное внешнее соединение}} \rightarrow S \bowtie_{\theta} R$	Перестановка входов соединения. Позволяет рассмотреть оба порядка вычисления.
2	Ассоциативность соединения	$\frac{(R \bowtie_{\theta_1} S) \bowtie_{\theta_2} T}{\text{оба соединения внутренние;} \theta_1 \wedge \theta_2 \text{ перераспределяемы}} \rightarrow R \bowtie_{\theta'_1} (S \bowtie_{\theta'_2} T)$	Изменяет группировку цепочки соединений. Открывает альтернативные порядки и уменьшает размер промежуточных результатов.
3	Проталкивание фильтра в левый вход	$\frac{\sigma_p(R \bowtie_{\theta} S)}{\text{attrs}(p) \subseteq \text{attrs}(R); \text{внутреннее соединение или сохраняющий вход}} \rightarrow \sigma_p(R) \bowtie_{\theta} S$	Применяет селективный предикат до соединения, уменьшая кардинальность входа.
4	Проталкивание фильтра в правый вход	$\frac{\sigma_p(R \bowtie_{\theta} S)}{\text{attrs}(p) \subseteq \text{attrs}(S); \text{внутреннее соединение или сохраняющий вход}} \rightarrow R \bowtie_{\theta} \sigma_p(S)$	Симметричный случай для правого входа.
5	Перенос фильтра в предикат соединения	$\frac{\sigma_p(R \bowtie_{\theta} S)}{\text{соединение внутреннее;} \text{attrs}(p) \subseteq \text{attrs}(R) \cup \text{attrs}(S)} \rightarrow R \bowtie_{\theta \wedge p} S$	Объединяет фильтр и предикат соединения. Может позволить применить хеш-соединение или соединение слиянием.
6	Преобразование декартова произведения в соединение	$\frac{\sigma_p(R \times S)}{\top} \rightarrow R \bowtie_p S$	Частный случай правила 5.
7	Поднятие фильтра над соединением	$\frac{\sigma_p(R) \bowtie_{\theta} S}{\text{attrs}(p) \subseteq \text{attrs}(R); \text{внутреннее соединение или сохраняющая сторона}} \rightarrow \sigma_p(R \bowtie_{\theta} S)$	Переносит фильтр выше соединения для объединения с предикатами родительских операторов.
8	Разбиение фильтра на конъюнкты	$\frac{\sigma_{p_1 \wedge \dots \wedge p_n}(R)}{\top} \rightarrow \sigma_{p_1}(\dots \sigma_{p_n}(R) \dots)$	Позволяет обрабатывать каждый конъюнкт независимо.
9	Проталкивание проекции через соединение	$\frac{\pi_L(R \bowtie_{\theta} S)}{\begin{array}{l} L_R \cup J_R \subsetneq \text{attrs}(R) \\ \text{или } L_S \cup J_S \subsetneq \text{attrs}(S) \end{array}} \rightarrow \pi_L(\pi_{L_R \cup J_R}(R) \bowtie_{\theta} \pi_{L_S \cup J_S}(S))$	Удаляет неиспользуемые атрибуты до соединения, сокращая ширину кортежей.

Продолжение таблицы 4

№	Название	Правило	Комментарий
10	Левое внешнее соединение во внутреннее	$\frac{\sigma_p(R \bowtie_{\theta} S)}{p \text{ null-отбрасывающий по } attrs(S)} \rightarrow$ $\sigma_p(R \bowtie_{\theta} S)$	Открывает для подплана правила, ограниченные внутренним соединением.
11	Полное внешнее соединение во внутреннее	$\frac{\sigma_p(R \bowtie_{\theta} S)}{p \text{ null-отбрасывающий по } attrs(R) \text{ и } attrs(S)} \rightarrow$ $\sigma_p(R \bowtie_{\theta} S)$	Требует null-отбрасывания с обеих сторон.
12	Проталкивание агрегации ниже соединения	$\frac{\gamma_G; F(R \bowtie_{R.k=S.k} S)}{F \text{ разделяема; } \text{внутреннее соединение по равенству}} \rightarrow$ $\gamma_G; F'(\gamma_{G_R \cup \{R.k\}}^{partial}; F_R(R) \bowtie_{R.k=S.k} S)$	Снижает кардинальность входа частичной группировкой. $G_R = G \cap attrs(R)$; F_R — частичные агрегаты по атрибутам R ; F' — финальные комбинаторы.
13	Перестановка агрегации и соединения	$\frac{\gamma_G; F_R(R \bowtie_{R.k=S.k} S)}{\text{внутреннее соединение по равенству; } F_R \subseteq attrs(R); R.k \in G; S.k \text{ уникален и полон}} \rightarrow$ $\gamma_G; F_R(R) \bowtie_{R.k=S.k} S$	Полностью переносит агрегацию до соединения. $S.k$ уникален: соединение не дублирует строки R . $S.k$ полон относительно $R.k$: соединение не теряет строки R .

Таблица 5 – Стоимостные формулы физических операторов.

Физический оператор	Формула стоимости
Последовательное сканирование	$100 n$
Фильтрация	$100 n_{out}$
Проекция	$22 n_{out}$
Сортировка	$11 n (\lfloor \log_2 n \rfloor + 1)$
Агрегация	$510 n$
Потоковая агрегация	$130 n$
Соединение вложенными циклами	$70 n_l n_r$
Соединение слиянием	$18 n_l + n_l + 10 n_o w_o$
Декартово произведение	$104 n_l n_r$
Хеш-соединение	$100 n_b w_b + 35 n_p + 10 n_o w_o$

Таблица 6 – Правила реализации.

№	Название	Правило	Комментарий
1	Последовательное сканирование	$\frac{\text{LogicalGet}}{\top} \rightarrow$ SeqScan	Применимо к любому отношению.

Продолжение таблицы 6

№	Название	Правило	Комментарий
2	Сканирование по индексу	$\begin{array}{c} \text{LogicalGet} \\ \xrightarrow{\text{есть совместимый индекс}} \\ \text{IndexScan} \end{array}$	Индекс должен быть совместим с предикатом или требуемым порядком.
3	Соединение вложенными циклами	$\begin{array}{c} R \bowtie_{\theta} S \\ \xrightarrow{\top} \\ \text{NestedLoopJoin} \end{array}$	Применимо при любом виде предиката. Стоимость высока без доступа по индексу к внутреннему отношению.
4	Хеш-соединение	$\begin{array}{c} R \bowtie_{\theta} S \\ \xrightarrow{\theta \text{ содержит равенство по ключам}} \\ \text{HashJoin} \end{array}$	Хеш-таблица строится по ключам равенства.
5	Соединение слиянием	$\begin{array}{c} R \bowtie_{\theta} S \\ \xrightarrow{\text{входы упорядочены по ключам соединения}} \\ \text{MergeJoin} \end{array}$	Требуемый порядок может быть обеспечен явным оператором Sort.
6	Хеш-агрегация	$\begin{array}{c} \gamma_{G; F}(R) \\ \xrightarrow{\top} \\ \text{HashAggregate} \end{array}$	Не требует упорядоченности входа. Потребляет память пропорционально числу групп.
7	Потоковая агрегация	$\begin{array}{c} \gamma_{G; F}(R) \\ \xrightarrow{\text{вход упорядочен по } G} \\ \text{StreamAggregate} \end{array}$	Эффективна при уже отсортированном входе. Может быть выгодна, если требуемый порядок нужен и родительскому оператору.

Листинг 16: Вычисление локальной стоимости физических операторов.

```

int64_t CalcCost(PhysicalExpr* expr, CardinalityEstimates& cardinality)
↪ {
    return std::visit(utils::Overloaded{
        [&](const physical::NestedLoopJoin& j) {
            return 70 * cardinality.GetCardinality(j.lhs)
                * cardinality.GetCardinality(j.rhs);},
        [&](const physical::HashJoin& j) {
            return 69 * (cardinality.GetCardinality(j.lhs)
                + cardinality.GetCardinality(j.rhs));}, ...
    }, *expr);
}

```

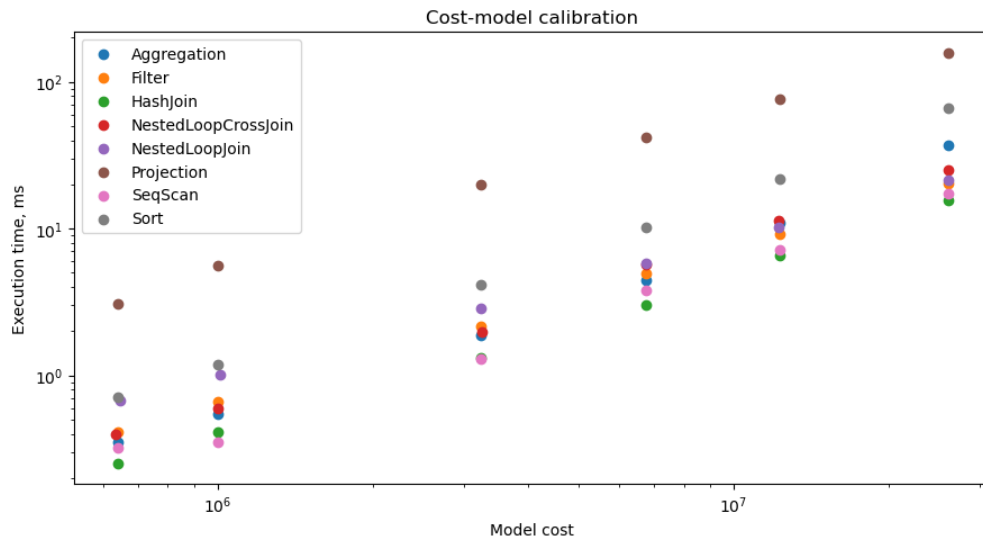


Рисунок 10 – Сопоставление расчетной стоимости физических операторов с фактическим временем выполнения.

Листинг 17: Правило коммутативности соединения.

```
LogicalOperator
↪ JoinCommutativity::ApplyImpl(utils::NotNull<LogicalExpr*> expr,
↪ Memo&) {
    auto join = std::get<logical::Join>(expr->root_operator);
    std::swap(join.lhs, join.rhs);
    if (join.type == JoinType::kLeft) {
        join.type = JoinType::kRight;
    } else if (join.type == JoinType::kRight) {
        join.type = JoinType::kLeft;
    }
    return join;
}
bool ImplementHashJoin::IsApplicable(utils::NotNull<LogicalExpr*> expr)
↪ {
    if (!std::holds_alternative<logical::Join>(expr->root_operator))
        return false;
    const auto& join = std::get<logical::Join>(expr->root_operator);
    if (join.type != JoinType::kInner) return false;
    const auto* bin = std::get_if<BinaryExpression>(&join.qual);
    return bin && bin->binop == BinaryOp::kEq &&
        ↪ std::holds_alternative<Attribute>(*bin->lhs) &&
        ↪ std::holds_alternative<Attribute>(*bin->rhs);
}
```

```
(.venv)
[st@nixos:~/c/lu9-sql-compiler]$ echo 'SELECT users.id FROM users;' | ./build/bin/sql --data-dir
test/static/executor/test_data
users.id:int
1
10
11
12
13
14
15
16
17
2
3
4
5
6
7
8
8
(.venv)
[st@nixos:~/c/lu9-sql-compiler]$
```

Рисунок 11 – Выполнение SQL-запроса над тестовыми данными

Листинг 18: Оптимизация группы Мемо.

```
void Optimizer::OptimizeGroup(Group* group, PropertySet required,
                             Limit limit) {
    WinnerKey key{group, required};
    if (winner_.contains(key)) return;
    if (IsExplored(group) && limit && LowerBoundCost(group) >= *limit)
        ↪ return;
    if (!IsExplored(group)) {
        tasks_.emplace( [= ] {
            OptimizeGroup(group, required, limit);
        });
        tasks_.emplace( [= ] {
            ExploreGroup(group, limit);
        });
        return;
    }
    for (auto expr : group->GetPhysicalExprs()) {
        if (expr->is_enforcer) continue;
        auto cost = local_cost_[expr.get()];
        if (!limit || cost < *limit) {
            tasks_.emplace( [= ] {
                OptimizeInputs(expr, required, {}, cost, limit);
            });
        }
    }
    AddEnforcers(group, required, limit);
}
```

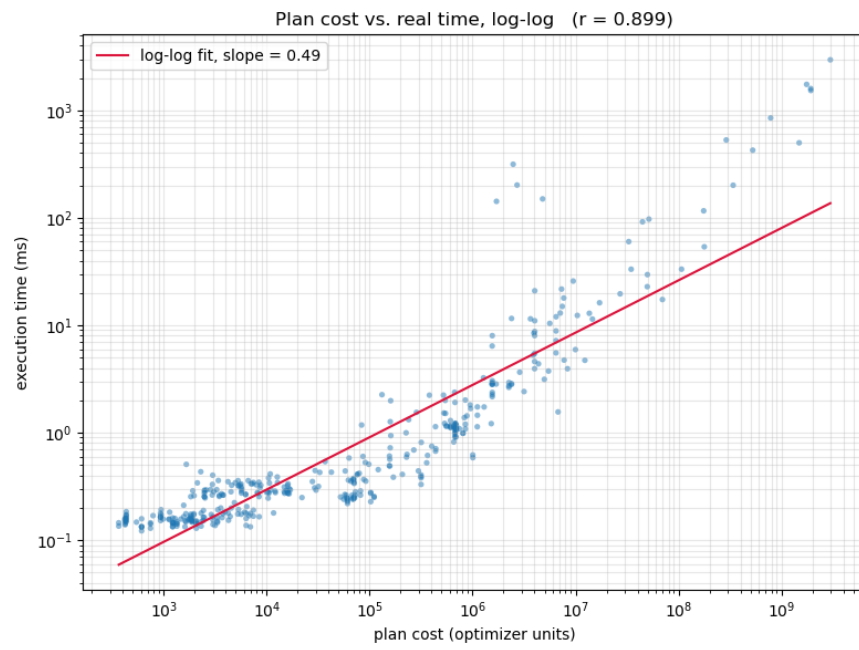


Рисунок 12 – Сравнение скорости выполнения и стоимости на случайных запросах.